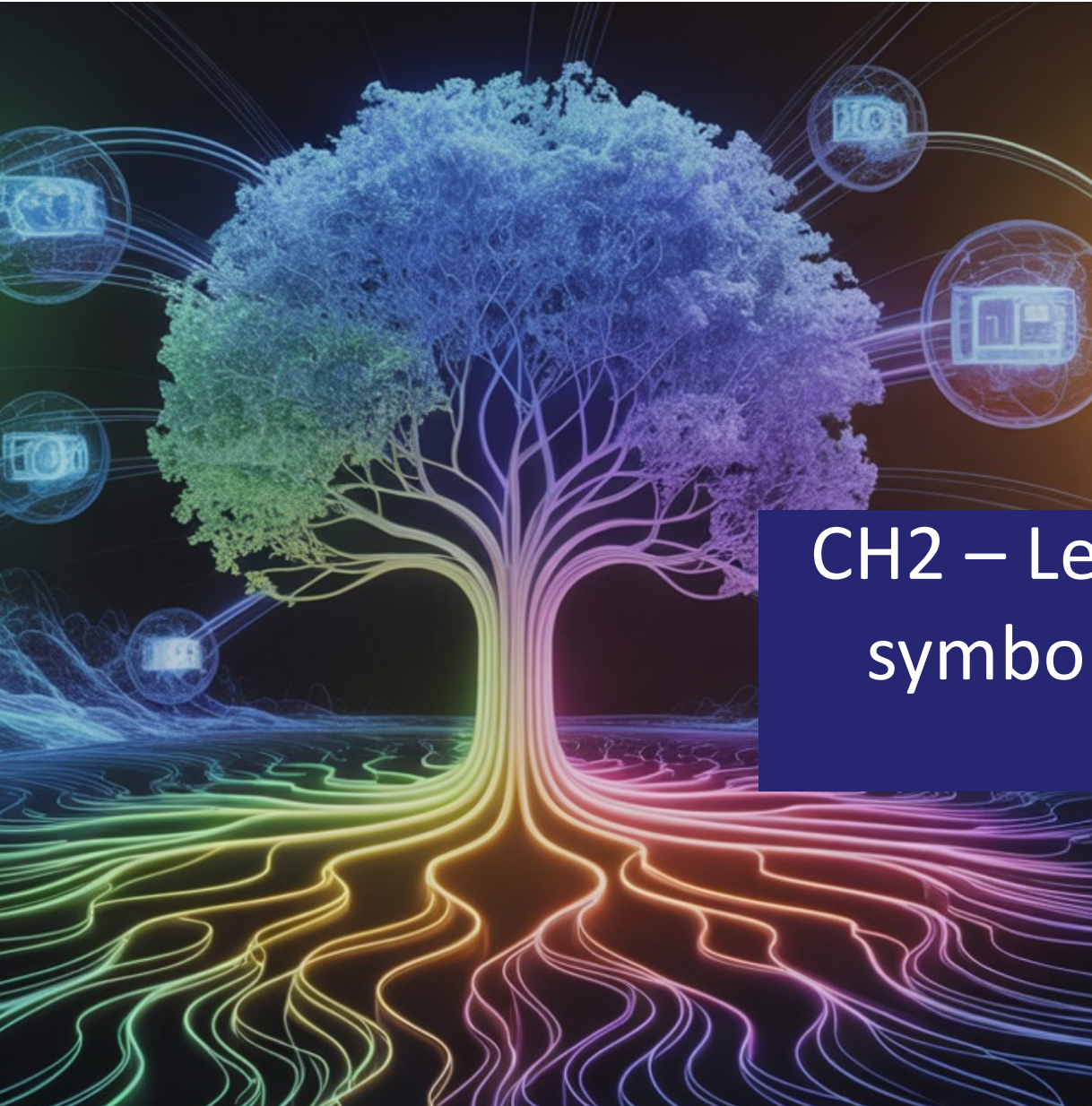




UNIVERSITE DE CORSE
Master Informatique
parcours DFS et DE
2^{ème} année
2025-2026

CH2 – Les représentations de l'IA symbolique : des origines aux ontologies

Evelyne VITTORI
vittori_e@univ-corse.fr

Partie II - Représenter le monde : des symboles aux vecteurs



CH1 – Représenter
le monde en IA : un
enjeu central



CH2 –
Représentations en
IA symbolique : des
origines aux
ontologies



CH3 – Représenter
par des vecteurs :
fondements des
embeddings

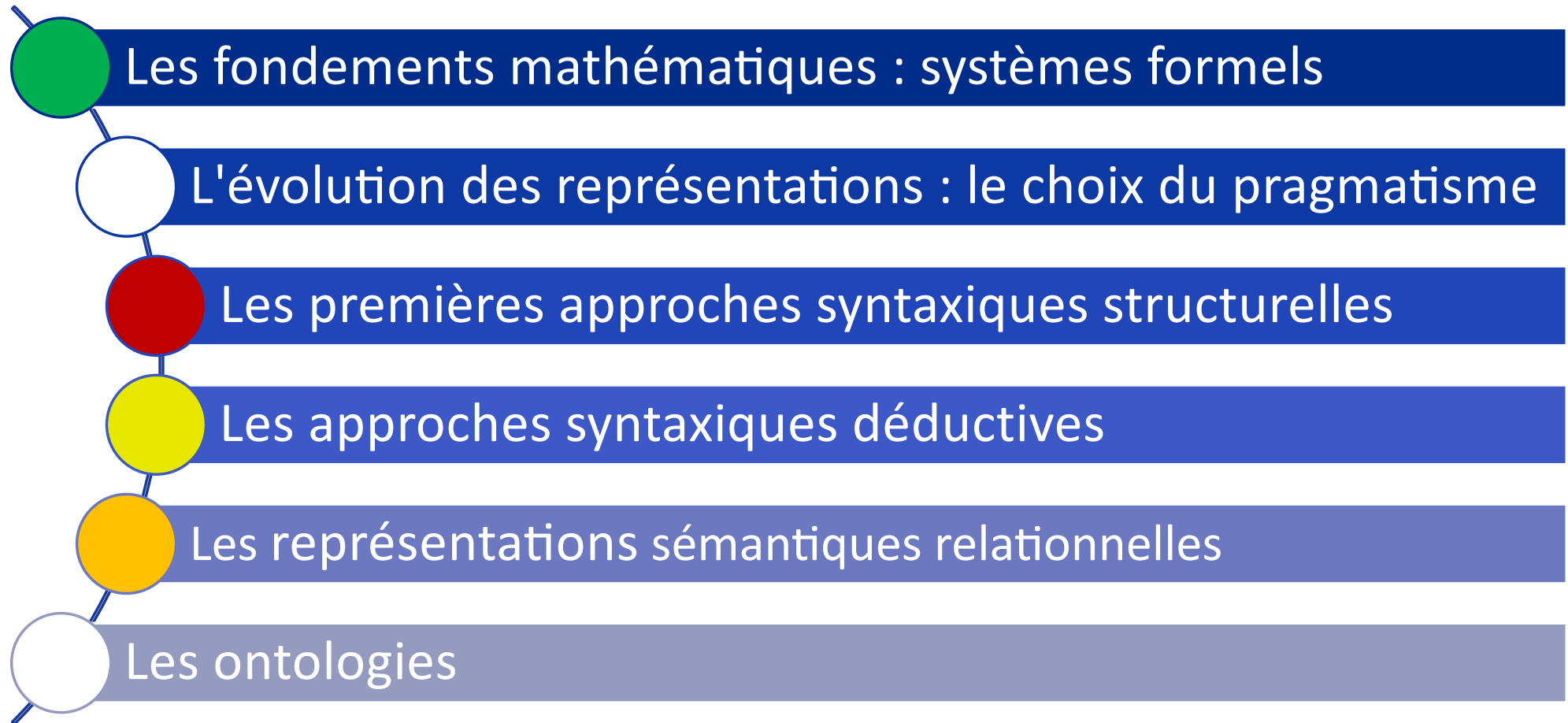


CH4 – Des
embeddings aux BD
Vectorielles

Objectifs du chapitre

- Comprendre les bases logiques et formelles des représentations symboliques
 - Identifier les principales approches syntaxiques et déductives de l'IA symbolique
 - Situer les représentations relationnelles non formelles (thésaurus, graphes, frames)
 - Découvrir les **ontologies** comme forme avancée de structuration du savoir
-  **Activité guidée:** Manipuler une ontologie simple avec **Protégé**

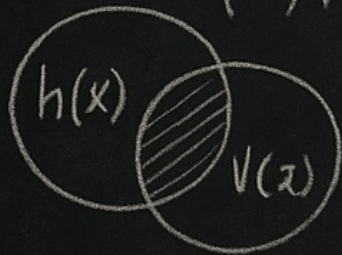
CH2 – Les représentations en IA symbolique: Des origines aux ontologies



1 - Les origines de l'IA symbolique : systèmes formels et logique

$$\sum_{U \cup V} \alpha(\tau(y) \wedge \nabla_\epsilon(t)) \geq y \quad (\exists x (vy) > 4 \sum_{n=1}^{\infty} f(f, a, \beta, y)$$

$$(\exists) \nabla \nabla > 4$$



$$\overline{U \cup V}$$

$$\forall, \nabla \cdot \nabla \cdot +$$

$$\perp \nabla, \nabla \cup \nabla \downarrow$$

$$f(x) = \begin{bmatrix} 4 & 0 & 1 \\ -3 & -1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

$$X = X / V +$$

$$gf(a, \beta) \quad g'(a) = f(x) = 3x^2 - 2x$$

$$\forall \cup \nabla \in \mathbb{R} \quad \prod_1(0, 1, 1, 0) = \overline{\mathbb{R}}$$

$$f'(x) = \frac{3x^1}{5} = 2x \frac{a}{a}$$

$$g(x) = \lim_{x \rightarrow a} \frac{f(x)h - f(x)}{h}$$

$$|A| < \infty, \quad A \cap C \in \mathbb{R}$$

$$f'(x) = \frac{3x^2 - 2^2x}{a}$$

$$g'(x) = \lim_{x \rightarrow a} \frac{f(x)h - f(x)}{h}$$

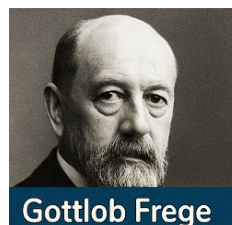


Les origines (avant 1930) :

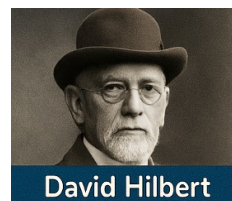
Les mathématiciens fondateurs de la logique formelle

Leurs objectifs :

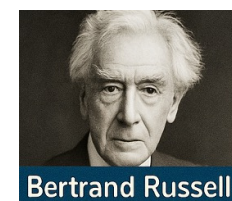
- **Éliminer l'ambiguïté** du langage naturel
→ Concevoir un langage formel
“*Begriffsschrift*” (1879)
- **Formaliser le raisonnement**
→ Énoncer toutes les règles de déduction logiques
“*système axiomatique*” (années 1920)
- Exprimer **tout le savoir mathématique...**
voire scientifique
→ Construire des systèmes formels complets et cohérents
“*Principia Mathematica*” (1910–1913)



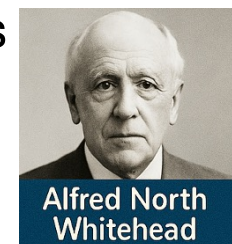
Gottlob Frege



David Hilbert



Bertrand Russell



Alfred North Whitehead



Vers un système formel
des mathématiques

Qu'est ce qu'un système formel ?



Alphabet

langage

- ensemble des symboles permettant de construire les formules



Formules Bien formées



Axiomes

- formules valides par définition



Règles d'inférence

- permettent de dériver de nouvelles formules : **les théorèmes**

Ex: Le système formel MIU (Hofstadter)



Alphabet : M, I, U



Formules Bien formées : toute suite de symboles de l'alphabet



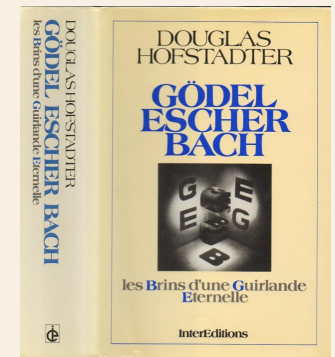
Axiomes : MI



Règles d'inférence :

Soient f et g des FBF

1. $fI \rightarrow fIU$ (règle de production)
2. $Mf \rightarrow Mff$ (règle de production)
3. $fIIIg \rightarrow fUg$ (règle de réécriture)
4. $fUUg \rightarrow fg$ (règle de réécriture)



Déduction dans un système formel

Ex: Le système formel MIU (Hofstadter)



Alphabet : M, I, U



Formules Bien formées : toute suite de symboles de l'alphabet



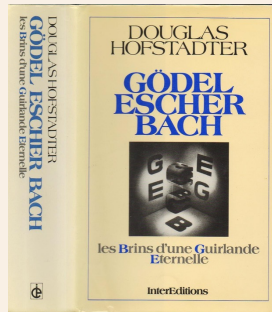
Axiomes : MI



Règles d'inférence :

Soient f et g des FBF

1. $fl \rightarrow fIU$ (règle de production)
2. $Mf \rightarrow Mff$ (règle de production)
3. $fIlg \rightarrow fUg$ (règle de réécriture)
4. $fUUg \rightarrow fg$ (règle de réécriture)



■ Exercice

Montrer que MUll est un théorème du système MIU

- MI par l'axiome
- MII par la règle 2
- MIII par la règle 2
- MIIIIIIII par la règle 2
- MIIIIIIIIU par la règle 1
- MIIIIIIUU par la règle 3
- MIIII par la règle 4
- MUll par la règle 3

Le système formel de référence : La logique des prédicats

FOL First Order Logic

■ Alphabet

Variables x, y	Constantes a, b, c
Fonctions $f(x)$ $g(x, y)$	Prédicats $P(x)$ $Q(x, y)$
Connecteurs logiques $\neg \wedge \vee \rightarrow$	Quantificateurs $\forall x$ $\exists x$

■ Formules bien formées

Exemple :

$P(a), \neg Q(x), \forall x (Aime(x, marie) \rightarrow Humain(x))$

■ Axiomes logiques : formules vraies par définition

- $(\exists x : \phi \rightarrow (\psi \rightarrow \phi))$

Si ϕ est vrai,
alors ψ implique ϕ

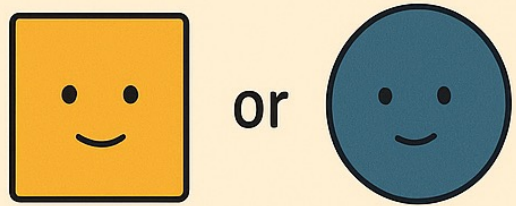
■ Règles d'inférence : permettent de dériver de nouvelles formules

- Exemple : **Modus ponens**

Si ϕ et $\phi \rightarrow \psi$,
alors on peut inférer ψ

Ce que FOL peut exprimer

$$\forall x (P(x) \vee Q(x))$$



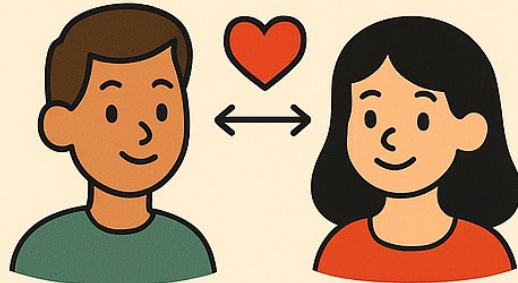
Tout a la propriété
P ou la propriété Q

$$R(\text{Alice}, x)$$



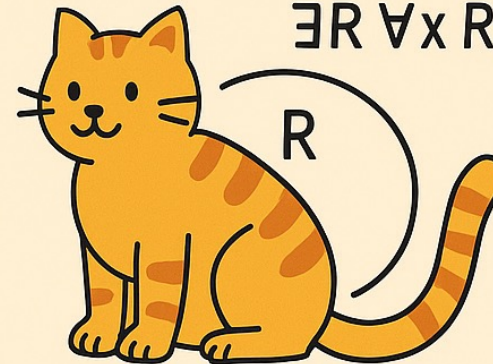
Alice est liée à quelqu'un

$$\forall x (M(x) \rightarrow \exists y L(y, x) \wedge L(x, y))$$



Être marié implique de
s'aimer l'un l'autre

$$\exists R \forall x R(x, x)$$



Il existe une relation
que tout a avec soi - même

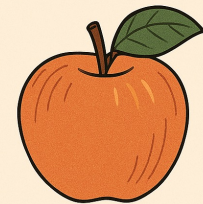
Le langage ne traduit pas toujours la réalité...



La Trahison des images ([René Magritte 1928–1929](#))

LES MOTS NE SONT
PAS LES CHOSES

pomme



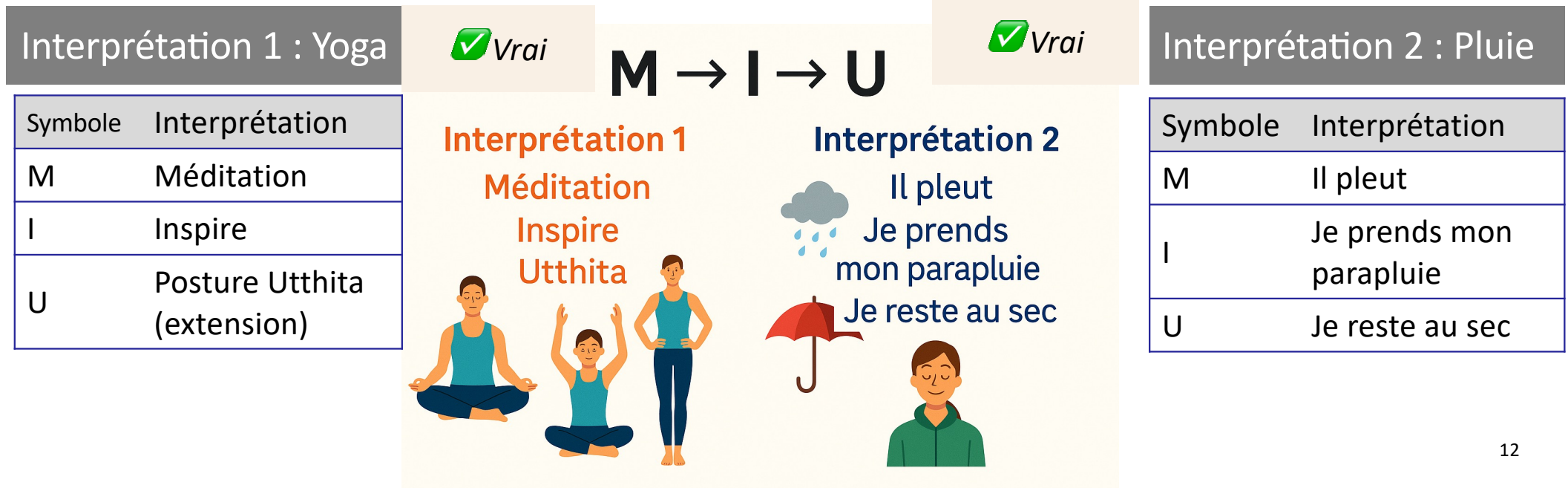
Ceci est une image,
pas une pomme

La syntaxe ne suffit pas

Donner un sens aux symboles : Sémantique d'un système formel

Définir une interprétation = définir un monde possible

1. Définir un domaine d'objets
2. Définir une affectation des symboles (constantes, fonctions, prédicats)

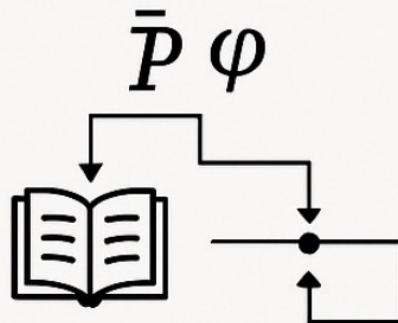


Prouvabilité (syntaxique) VS Vérité/Validité (sémantique)

- Une formule est **prouvable** si elle est démontrable syntaxiquement

Théorème

Syntaxe pure



Démonstration
à partir d'axiomes

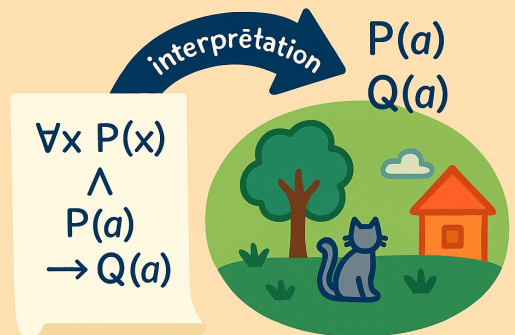
- Une formule est "**vraie**" si elle est vérifiée dans une interprétation
- Une formule est "**valide**" si elle est vraie dans toutes les interprétations

Sémantique
(vérité relative à une
interprétation)



Évaluation dans
des mondes possibles

Sémantique d'un système formel



Prouvabilité vs Validité

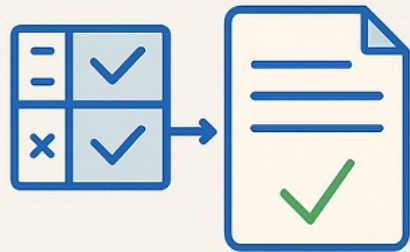


Alfred TARSKI 1933

Théorie de la preuve (axiomatique)	Théorie des modèles (sémantique)
 Syntaxe pure	 Sémantique (vérité relative à une interprétation)
Démonstration à partir d'axiomes dans un système formel fermé S	Évaluation dans une Interprétation = mondes possibles – un domaine d'objets – une affectation des symboles (constantes, fonctions, prédicats)
Manipule des formules sans "sens" (notion de démontrabilité)	Donne un sens aux symboles dans une interprétation (notion de vérité relative)
$\vdash \phi$: ϕ est prouvable (démontrable à partir d'axiomes) : ϕ est un théorème de S	$I \models \phi$: ϕ est vraie dans l'interprétation I Une interprétation I est un modèle d'une formule ϕ si ϕ est vraie dans I
	$\models \phi$: ϕ est valide si elle est vraie dans toutes les interprétations possibles  Toutes les interprétations sont alors des modèles de ϕ

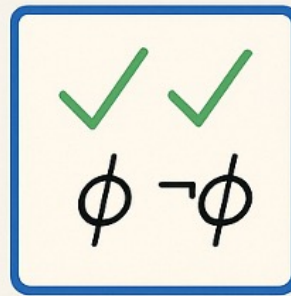
Propriétés des systèmes formels

COMPLÉTUDE



Toute formule
valide est
démontrable

COHÉRENCE



Il n'existe aucune
formule ϕ et $\neg\phi$
qui soient toutes
les deux
démonstrables

DÉCIDABILITÉ



Il existe un
algorithme qui
peut décider si
une formule est
démontrable

Le théorème de complétude de FOL (Gödel, 1929)

Dans le système formel de la logique des prédicats (FOL)
Toute formule valide est prouvable (et inversement) : $\models \phi \Leftrightarrow \vdash \phi$

Adéquation syntaxe / sémantique :

- Il existe une correspondance parfaite entre vérité dans tous les modèles (sémantique) et preuve à partir d'axiomes (syntaxe)

Base des systèmes automatisés de preuve :

- Permet de créer des moteurs d'inférence complets pour la logique du premier ordre
-  Fondement des systèmes experts, Prolog, reasoners OWL

Après la logique FOL : des théories pour formaliser toute la mathématique

Théorie définie à partir de FOL:

- Langage formel spécifique défini à partir du langage de FOL et adapté au domaine : (variables, symboles, quantificateurs...)
- Axiomes spécifiques



Mais ces théories ont des limites

Propriétés des théories formelles

■ Récursivement axiomatisable

- Il existe un algorithme capable **d'énumérer tous les axiomes** de la théorie, un par un
- L'algorithme **ne s'arrête jamais**, mais si une formule est un axiome, elle finira par apparaître dans la liste

■ Décidable

- Il existe un algorithme qui décide, pour toute formule, si elle est un théorème
- L'algorithme s'arrête toujours avec une réponse correcte

Théorie	Récursivement axiomatisable ?	Décidable ?
Logique propositionnelle	✓ Oui	✓ Oui
FOL	✓ Oui	✗ Non
PA Arithmétique de Peano	✓ Oui	✗ Non

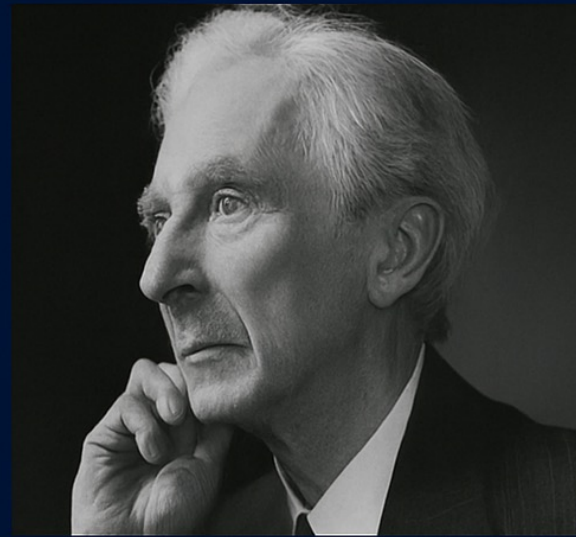
Toute théorie décidable est récursivement axiomatisable

L'espoir d'une IA symbolique

Nous croyions pouvoir tout modéliser à l'aide d'un système formel (axiomes + règles)



David Hilbert



Bertrand Russell

Gödel a brisé ce rêve en 1931

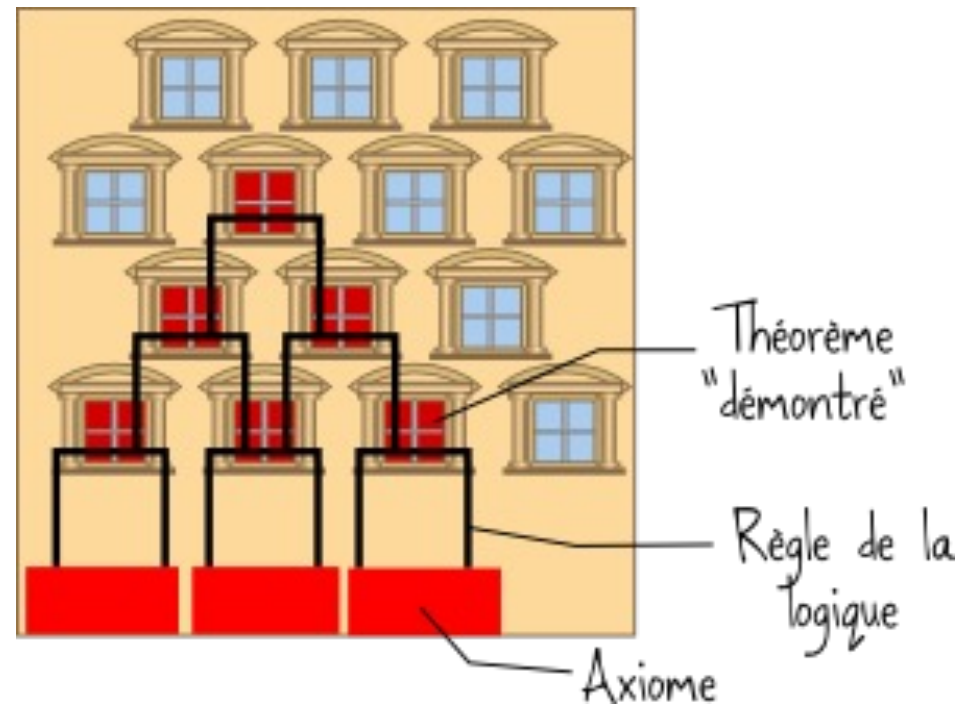
Théorème d'incomplétude (Gödel, 1931)

Dans toute théorie formelle suffisamment expressive pour formaliser l'arithmétique :

Il existe des énoncés vrais mais qu'il est impossible de démontrer



INCOMPLÉTUDE



Il est impossible de construire un système de preuve automatique complet pour les mathématiques

Travaux de Church et Turing (1936)

Il n'existe pas d'algorithme général pour déterminer si une formule est un théorème dans :

- La logique FOL elle-même
- Les théories récursivement axiomatisables comme PA, ZFC, etc



INDECIDABILITE



Récursivement axiomatisable :

Il existe un algorithme
pour énumérer tous les axiomes



Alonzo CHURCH



Alan TURING

2 – L'évolution des représentations : le choix du pragmatisme



Conséquences pour l'IA symbolique



Plutôt que tout exprimer, on restreint le langage !

Logique de Horn (base des systèmes experts et Prolog)



Décidable, résolution efficace, raisonnement déductif

Logiques de description (DL) (base d'OWL, Protégé)



Raisonnement automatique possible (via reasoners), base du Web sémantique

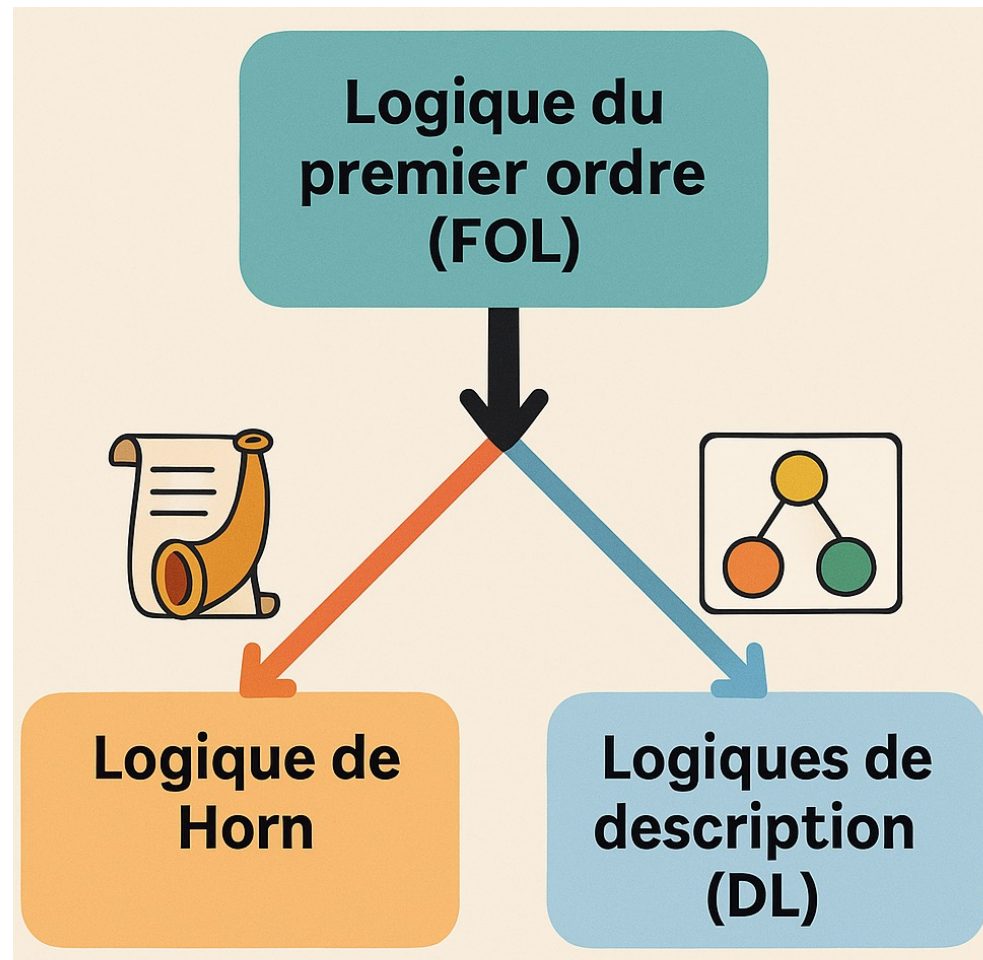
On choisit des logiques limitées mais exploitables automatiquement
Compromis puissance expressivité / faisabilité algorithmique

Les restrictions de FOL pour l'IA symbolique

 **Semi-Décidable**
: Peut ne jamais s'arrêter
en cas d'échec

- Implications simples :
une tête $\leftarrow$ un corps

 Applications :
Prolog, systèmes experts

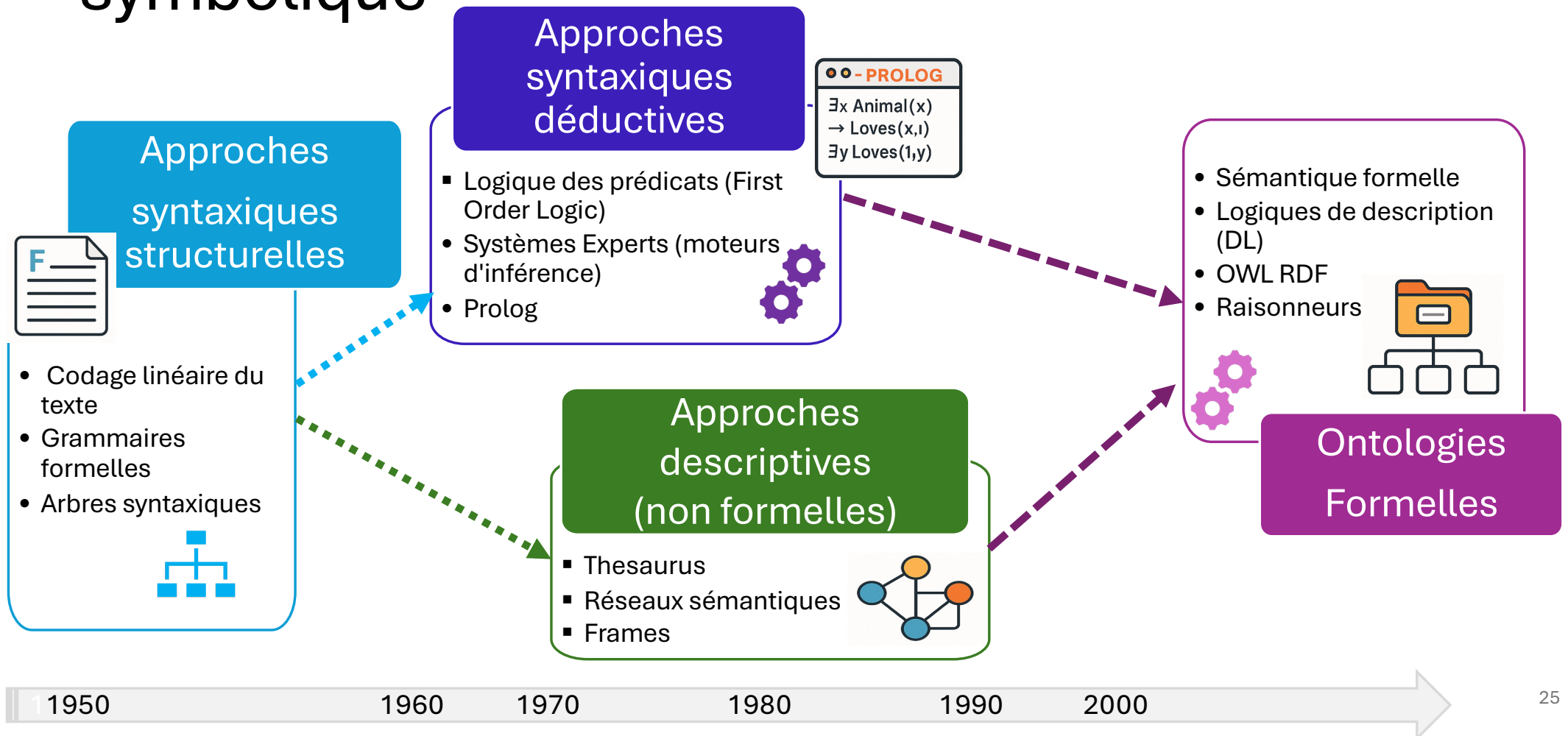


 **Décidables**

- Pas de quantificateurs imbriqués
- Seulement 1 variable libre à la fois
- Usage restreint des connecteurs ET OU
- Relations unaires

 Applications :
Ontologies, Reasoners

Evolution des représentations du savoir en IA symbolique



Pour clarifier: une vidéo très pédagogique

- Le théorème de Gödel | Voyages au pays des maths | ARTE



- <https://www.youtube.com/watch?v=Brj1LC42vLM>

Il reste encore des énigmes ...

Si la phrase "tous les crétois sont des menteurs" est vraie, alors ce qu'elle dit est vrai...

👉 donc elle est fausse

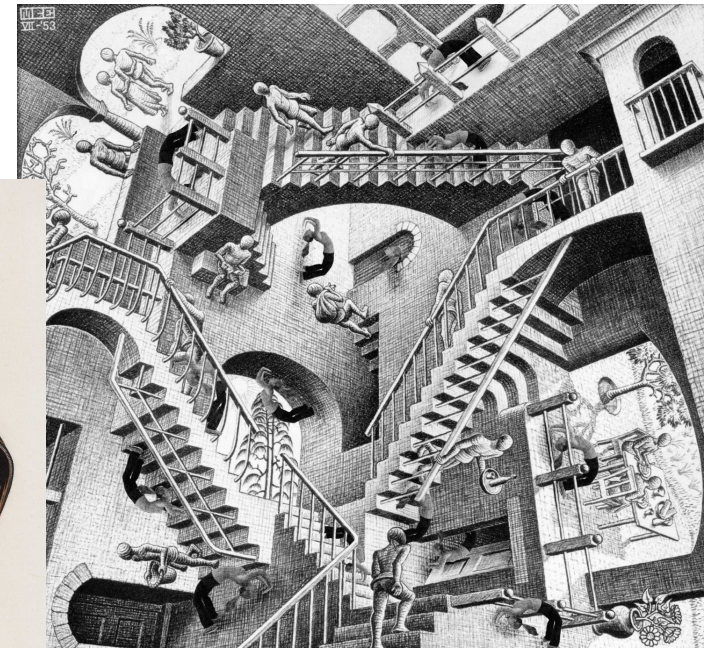
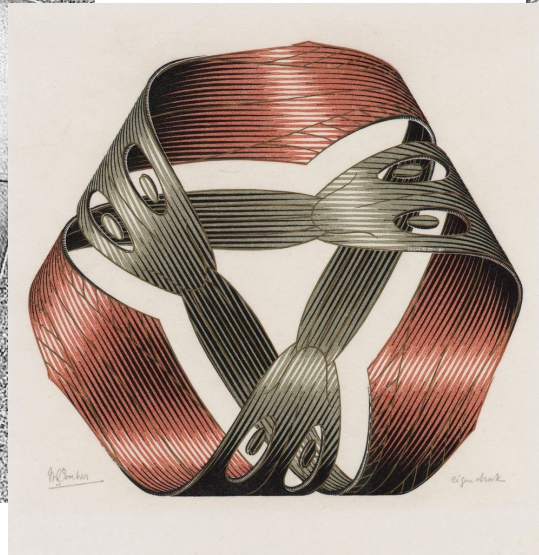
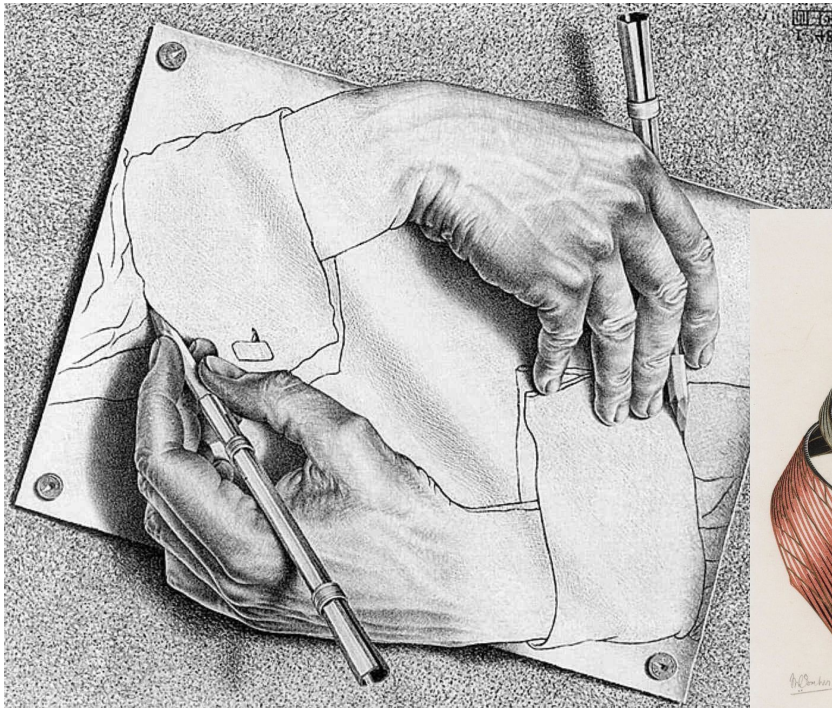
Si elle est fausse, alors ce qu'elle dit est faux...

👉 donc elle est vraie

🔄 **Boucle logique** : l'énoncé ne peut être ni vrai ni faux sans contradiction



Il reste encore des énigmes : où est la vérité ?



Boucles étranges

3 - Les représentations syntaxiques structurées



Une approche syntaxique minimale (1950...)

Codage historique du texte

- Chaînes de caractères (strings)
- **Chaque** caractère est représenté par un code numérique (ex : ASCII, Unicode)
- Le texte est une suite ordonnée de symboles, sans analyse sémantique
- **Exemple**
 - Chaîne : "Chat"
 - Représentation ASCII : [67, 104, 97, 116]
 - En binaire : 01000011
01101000 01100001
01110100

Représentation linéaire des mots
uniquement syntaxique, sans accès à la signification

Modélisations syntaxiques structurées (1956 – 1960)

■ Introduction des grammaires formelles

- Noam Chomsky, 1956 : grammaires génératives
- Modèles utilisés pour reconnaître et générer des phrases grammaticales

 On retrouve ici des concepts familiers de la compilation : grammaires hors-contexte, analyse syntaxique, arbres de dérivation

■ Arbres syntaxiques

- Représentation visuelle de la structure grammaticale interne
- Mise en évidence de la hiérarchie entre les éléments (sujet, verbe, complément...)

Représentation de phrases grammaticalement correctes, mais toujours sans accès à leur sens

Grammaires context-free (CFG)

- Règles syntaxiques décrivant la structure des phrases
- Utilisables pour la reconnaissance et la génération de phrases correctes
- Phrases conformes à cette grammaire :
 - *"Le chat dort"*
 - *"Un enfant regarde la souris"*
 - *"Un chien dort sur le tapis"*

Exemple de Règles

```
S → NP VP
NP → Det N | N
VP → V NP | V PP | V
PP → P NP
Det → "le" | "la" | "un" | "une"
N → "chat" | "chien" | "tapis" | "souris" | "enfant"
V → "dort" | "mange" | "regarde" | "aime"
P → "sur" | "sous" | "avec"
```

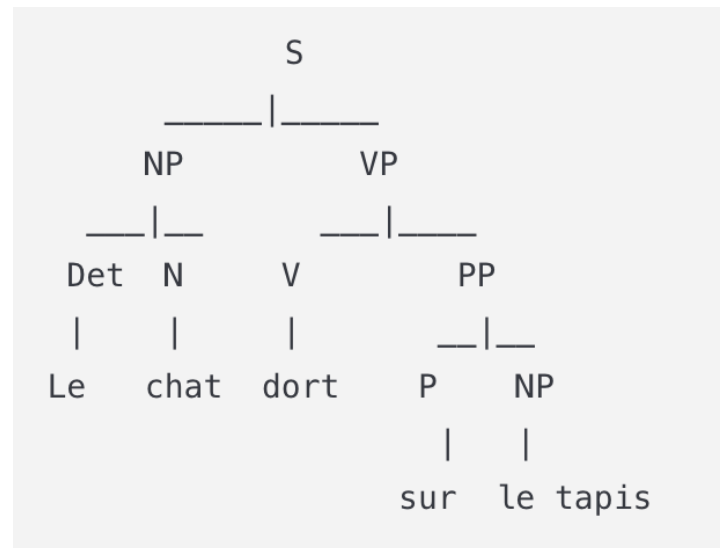
N (Noun) → Nom
V (Verb) → Verbe
P (Preposition) → Préposition
Det (Determiner) → Déterminant

SS (Sentence) → Phrase
NP (Noun Phrase) →
Groupe nominal (GN)
VP (Verb Phrase) →
Groupe verbal (GV)
PP (Prepositional Phrase) →
Groupe prépositionnel (GP)

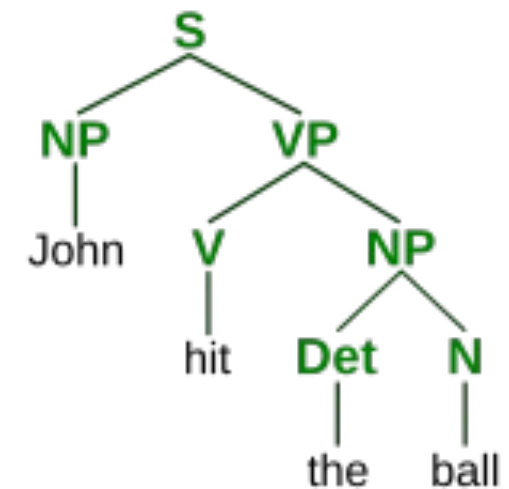
Arbres syntaxiques

- L'arbre décrit la structure grammaticale
- La compréhension passe par l'analyse hiérarchique des constituants

Phrase analysée :
"Le chat dort sur le tapis"



Phrase analysée : "John hit the ball"



source: https://en.wikipedia.org/wiki/Parse_tree

4 - Les approches syntaxiques déductives



Approches syntaxiques déductives

- Restriction de la Logique des prédicats (FOL) aux clauses de Horn

Sous-ensemble complet

Semi-décidable

- Automatisation du raisonnement
- Langage PROLOG

Une clause de Horn contient **au plus un littéral positif**

Formule de de la forme

$(A \wedge B) \Rightarrow C$ soit $\neg A \vee \neg B \vee C$

En prolog: règle $C :- A, B$

Systemes experts

Comprendre et reproduire la structure du langage humain (syntaxe) par une modélisation explicite et manipulable

Qu'est-ce qu'un système expert ?

Ensemble de faits et de règles de la forme :

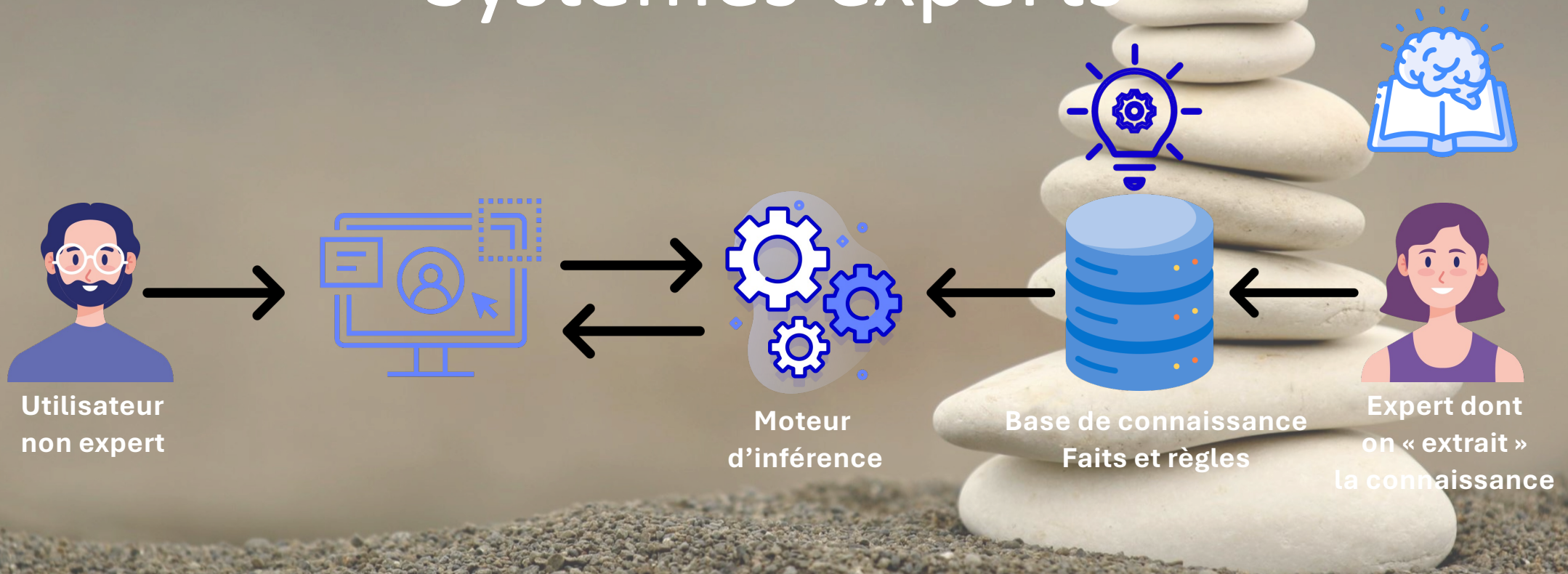
Si [une certaine condition est réalisée], alors
tirer [une certaine conclusion, avec un certain
degré de probabilité]

Les règles sont dictées par des experts du domaine c'est la phase
d'élicitation ou d'acquisition des connaissances

Un **moteur d'inférence**, permet à partir des faits et des règles de
produire de nouveaux faits

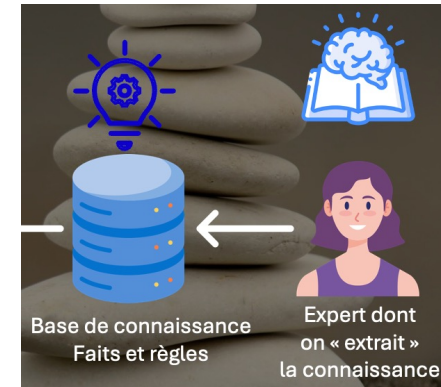
Ingénierie de la connaissance

Systemes experts



Aide à la décision

Systemes experts, extraction des connaissances



- **Règle 1 : Si** la température du patient est supérieure à 38°C **et** s'il a des frissons, **alors** le patient pourrait avoir une infection
- **Règle 2 : Si** un patient est âgé de plus de 60 ans **et** présente une toux persistante, **alors** il pourrait avoir une pneumonie
- **Règle 3 : Si** le patient a des douleurs abdominales **et** des nausées, **alors** il pourrait souffrir d'une gastrite
- **Règle 4 : Si** le patient a des douleurs abdominales aiguës, **et** localisées dans la partie inférieure droite, **alors** il pourrait souffrir d'une appendicite

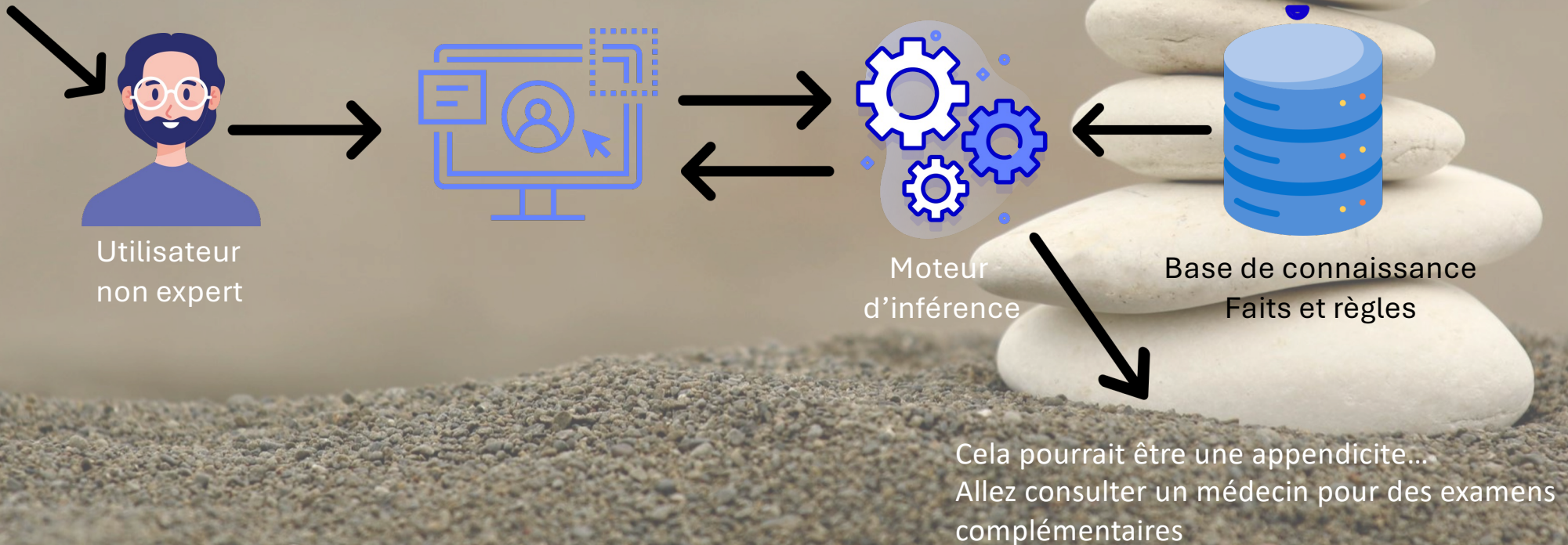
Systemes experts, inférence de nouveaux faits

Fait 1 : Le patient est âgé de 45 ans

Fait 2 : Le patient présente une température de 39°C

Fait 3 : Le patient souffre de nausées

Fait 4 : Le patient souffre de douleurs abdominales à droite



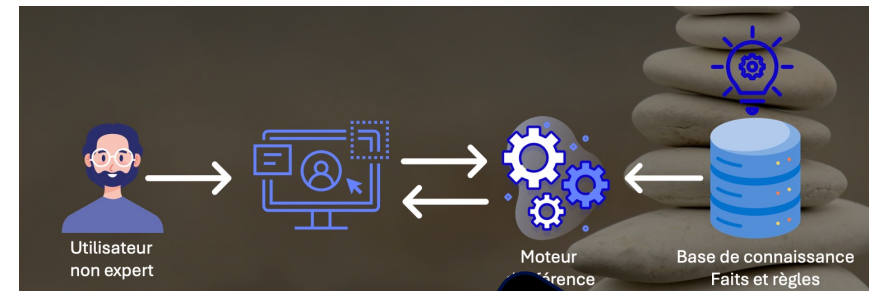
Systemes experts,

Utilisation du moteur d'inférence

- **Fait 1** : Le patient est âgé de 45 ans
- **Fait 2** : Le patient présente une température de 39°C
- **Fait 3** : Le patient souffre de nausées
- **Fait 4** : Le patient souffre de douleurs abdominales à droite

- **Règle 1** : Si la température du patient est supérieure à 38°C et s'il a des frissons, alors le patient pourrait avoir une infection
- **Règle 2** : Si un patient est âgé de plus de 60 ans **et** présente une toux persistante, alors il pourrait avoir une pneumonie
- **Règle 3** : Si le patient a des douleurs abdominales **et** des nausées, alors il pourrait souffrir d'une gastrite
- **Règle 4** : Si le patient a des douleurs abdominales aiguës, et localisées dans la partie inférieure droite, alors il pourrait souffrir d'une appendicite

→ Règles non connues de l'utilisateur non expert, mais utilisables a posteriori pour **comprendre** les conclusions du système



Good Old Fashion AI
(GOFAI)

Explicabilité

Pourquoi cette approche a échoué dans les années 1990 : 2^{ème} hiver de l'IA

- *Expressivité limitée et manque de naturel*
 - Difficile de représenter des connaissances floues, incomplètes, contextuelles ou évolutives
- *Rigidité des règles*
 - Tout savoir devait être formalisé sous forme de faits ou de règles logiques, ce qui demande un effort considérable
- *Problèmes d'échelle*
 - Explosion combinatoire : la taille des bases de règles devient vite ingérable
- *Dépendance à l'expert humain*
 - Les règles sont écrites à la main : risque de subjectivité, coût élevé, couverture partielle

Nécessité de descriptions plus naturelles et plus souples

Les critiques philosophiques

Exemples : Critiques publiées en 1964 et 1972 par le philosophe Hubert Dreyfus

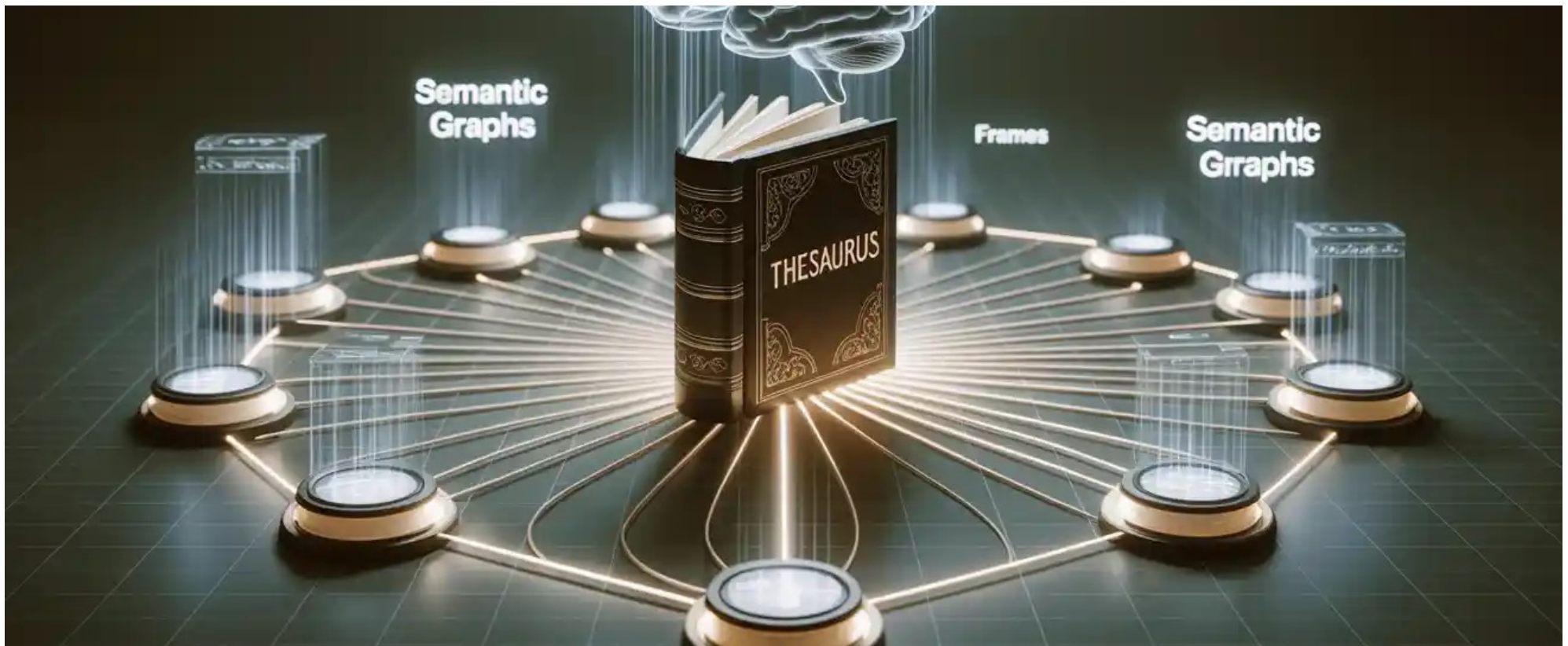
- « *Les premiers pas de l'IA dans les années 50/60 sont comparables au fait de grimper dans un arbre pour atteindre la lune* »

What computers can't do (1972, traduit en français sous le titre "Intelligence Artificielle, mythes et limites")



L'intelligence ne peut être réduite à une manipulation de symboles !

5 - Les représentations sémantiques relationnelles (non formelles)



Thesaurus : Indexer le sens avec des listes de mots et de relations

- Notion de thésaurus : mots clés + relations sémantiques
- Utilisés dans les systèmes de recherche documentaire
- Définition explicite par l'expert des relations entre concepts :
 - synonymie,
 - généralisation/spécialisation
 - association thématique (ex : "chat" relié à "animal domestique")

Exemple de thesaurus simplifié

Chat

- ├ Terme générique : Mammifère
- ├ Synonymes : Félin domestique
- ├ Termes spécifiques : Chat siamois, Chat persan
- ├ Associé à : Souris, Litière, Ronronnement

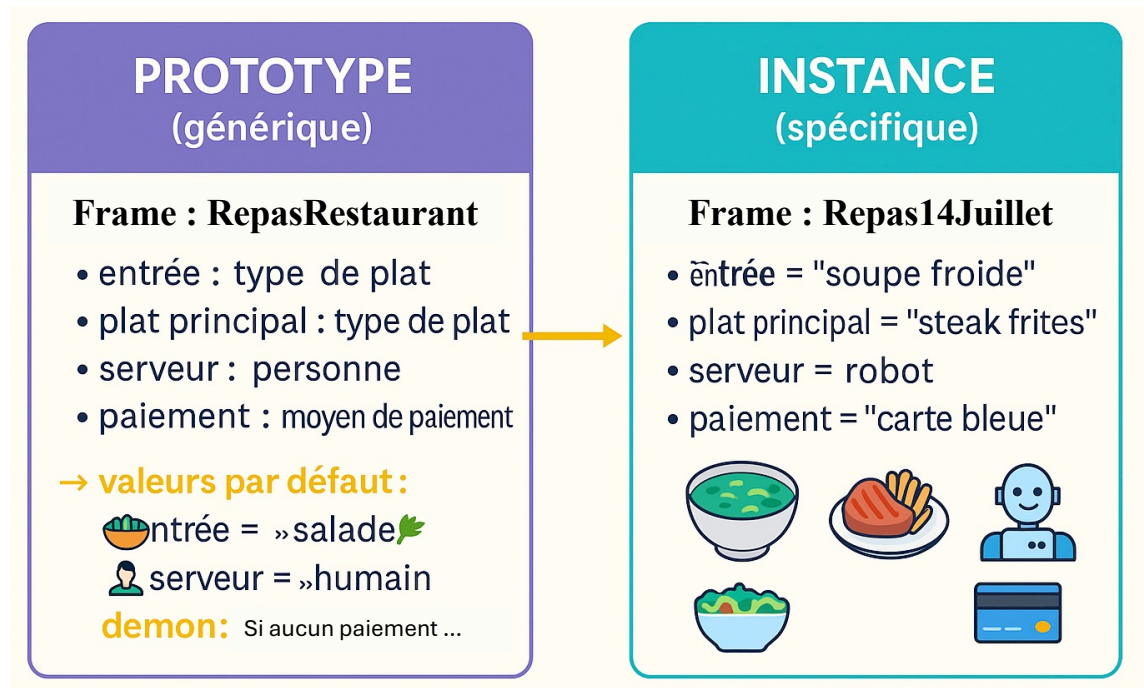
Nombre de relations limité, pas d'axiomes formels permettant le raisonnement automatique

Frames (Minsky 1974)



- Structures de **représentation des connaissances** avec procédures
 - Nom du frame
 - **Slots** : attributs
 - **Aspects** de description :
 - types, Valeurs par défaut, restrictions, contraintes ...
 - Procédures attachées (**démons**) : actions automatiques associés aux slots
 - Relations : is-a, part-of, instance-of
- Types de frames
 - Prototype : modèle générique
 - Instance : occurrence concrète

■ Exemple

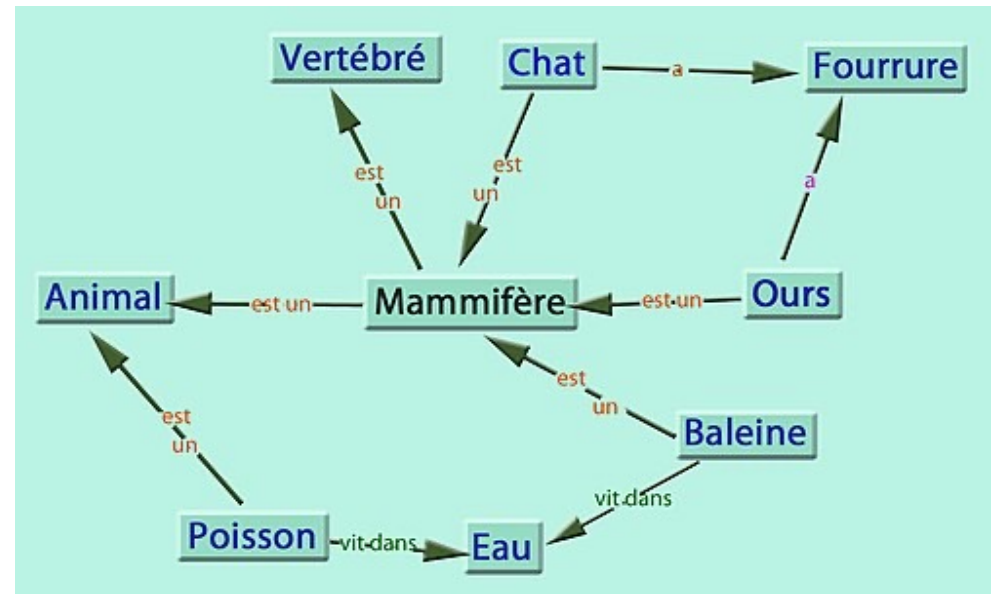


Faible formalisation: inférences difficiles

Article de référence [Minsky 1974] : <https://courses.media.mit.edu/2004spring/mas966/Minsky%201974%20Framework%20for%20knowledge.pdf>

Les réseaux sémantiques : premiers modèles graphiques des connaissances

- Structure de graphes avec
 - **nœuds** = **concepts** et
 - **arcs** = **relations sémantiques** (is-a, part-of, etc)
- Représentation visuelle intuitive des relations entre concepts
- Limites : **formalisation limitée**

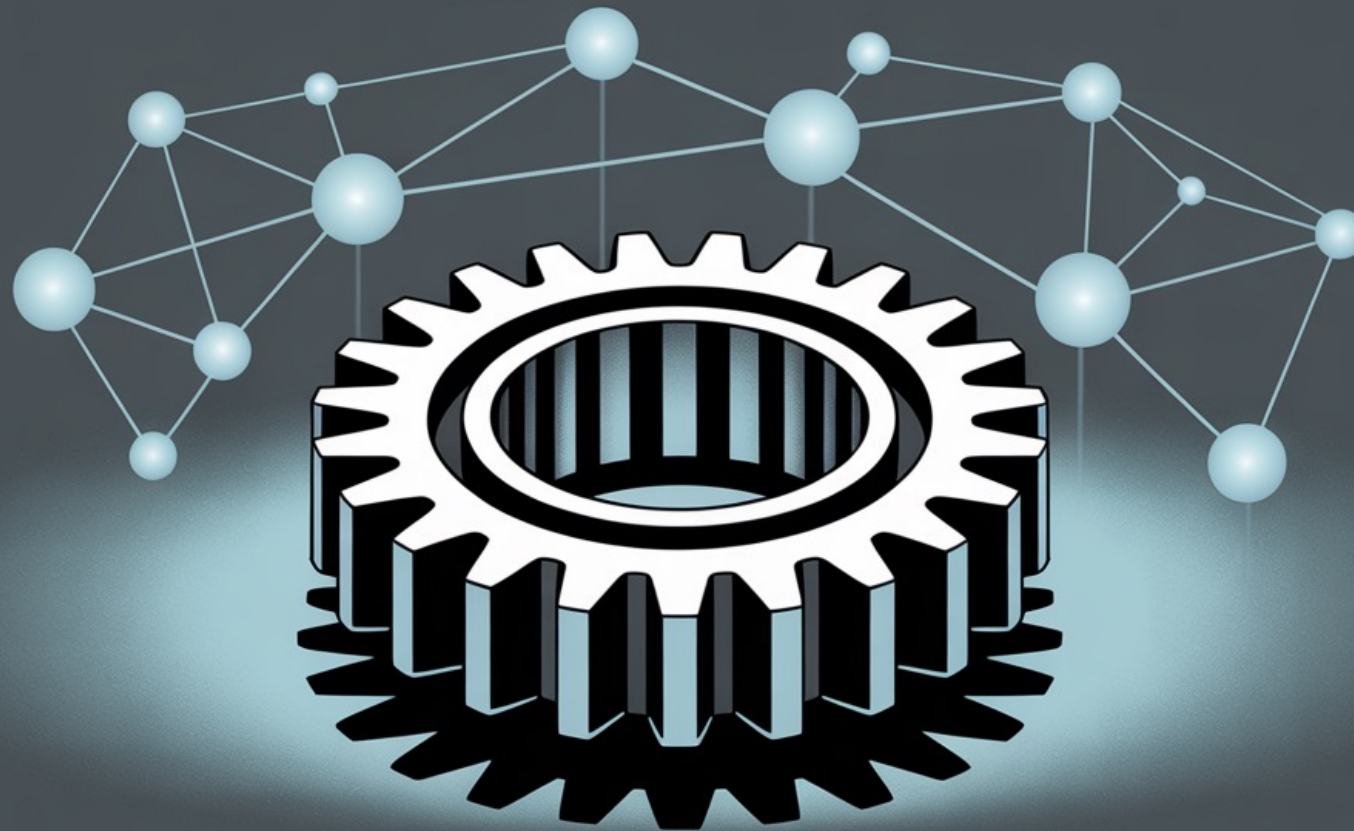


source: https://fr.wikipedia.org/wiki/R%C3%A9seau_s%C3%A9mantique

Etape importante pour passer d'une simple liste de termes (thesaurus) à une modélisation plus riche (ontologie)

Ref: J F Sowa, "Chapter 5 Conceptual Graphs," in *Foundations of Artificial Intelligence*, vol 3, Elsevier, 2008, pp 213–237 doi: [10 1016/S1574-6526\(07\)03005-2](https://doi.org/10.1016/S1574-6526(07)03005-2)

6 - Les Ontologies formelles : Aller plus loin dans la modélisation des liens entre concepts



Découvrir les Ontologies formelles

Introduction aux ontologies

- Origines et Définition
- Composants d'une ontologie
- Ontologies et Web Sémantique
- Applications et Types d'ontologies
- Objectifs

1



Langages et standards pour les ontologies

- Les formalisme RDF, Turtle, RDFS
- Les **logiques** de description
- Le langage OWL
- Le langage de requête SPARQL
- Outils et plateformes

2



Exploitation opérationnelle d'une ontologie

(exemple)

4



Construction d'une ontologie avec Protégé



Activité guidée

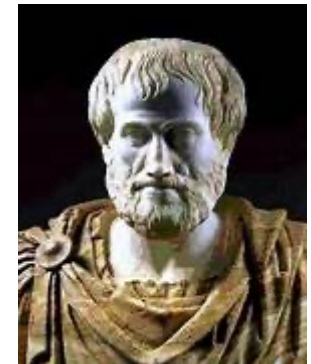
3

Origines des Ontologies

Philosophie

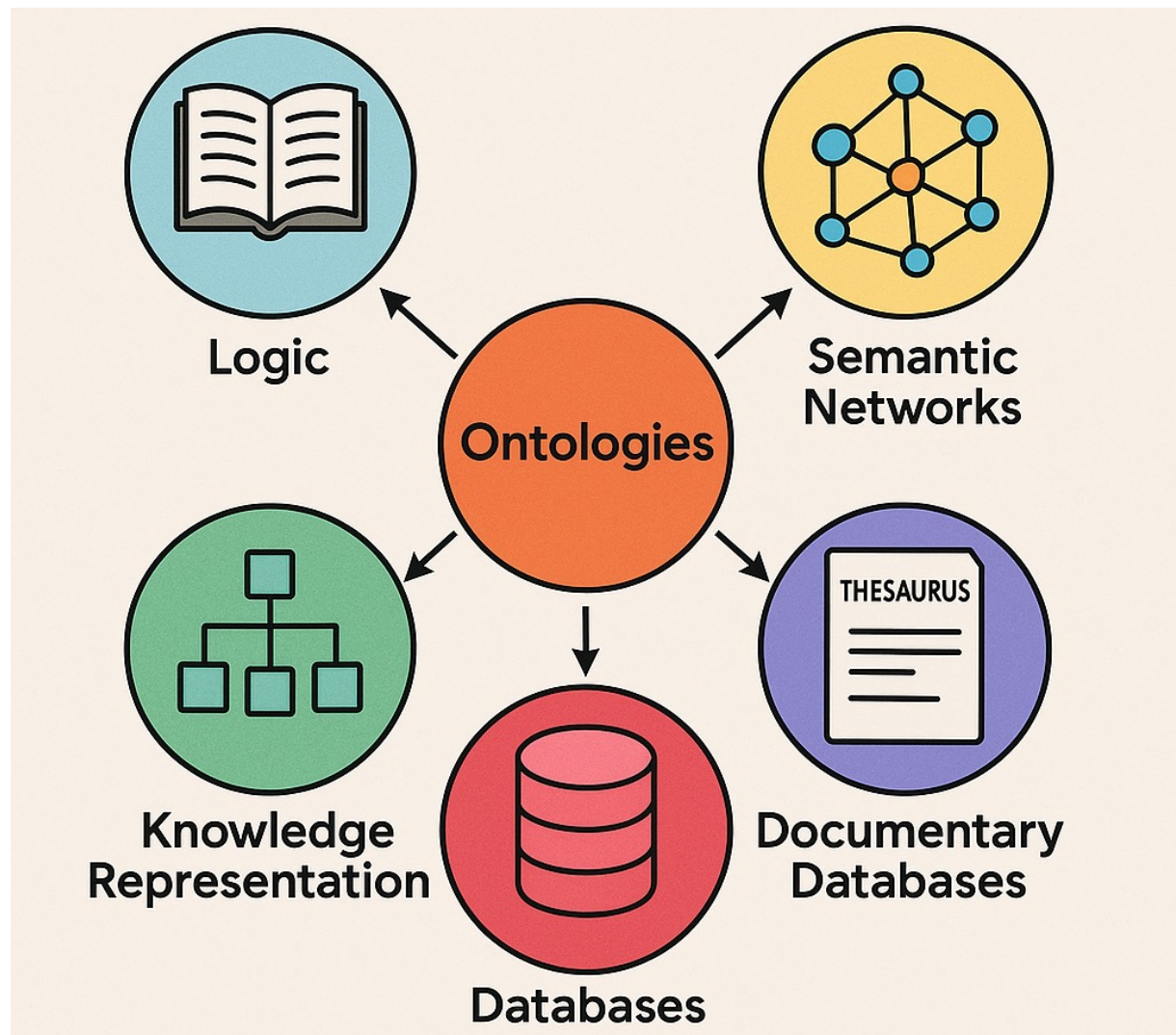
- Etude philosophique de l'être et des concepts associés tels que la réalité, l'existence et le devenir
- *"L'ontologie est la science de l'être en tant qu'être"*

Qu'est-ce que l'être ? Existe-t-il une réalité indépendante de notre perception ? Comment les entités se définissent-elles par rapport à un tout ?



Aristote

Ontologies en informatique



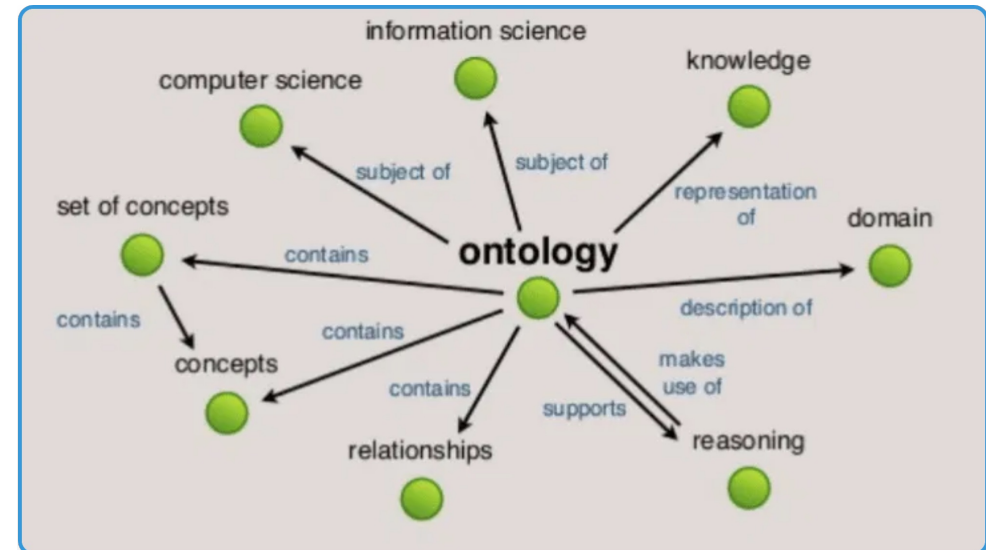
Qu'est-ce qu'une ontologie ?

- **Définition (Gruber, 1993)**

"Une ontologie est une spécification explicite d'une conceptualisation"



Tom Gruber
Co-createur de Siri



- **Modèle formel** décrivant explicitement des concepts, leurs relations, propriétés, axiomes et contraintes logiques

Composants Clés d'une Ontologie



TBox (Terminological Box)

Classes (concepts)

- Ensembles, collections ou types d'objets qui partagent des propriétés communes
- Exemple : "Personne", "Bâtiment", "Œuvre d'art"

Propriétés (relations)

- Liens entre individus ou classes, ou attributs caractérisant les individus
- Organisation hiérarchique des classes
- Exemple : "estAuteurDe", "estSituéÀ", "aUneHauteurDe"



Axiomes (Règles)

- Assertions, contraintes et règles logiques qui définissent les relations entre les concepts et permettent le raisonnement



ABox (Assertional Box)



Individus (instances)

- Objets concrets ou abstraits qui représentent les entités spécifiques du domaine
- Exemple : "Socrate", "Tour Eiffel", "Mona Lisa"

Ontologies et Web Sémantique



Web classique



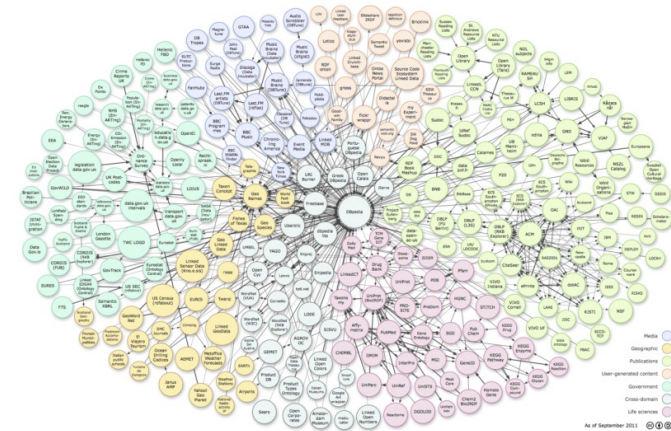
Recherche par mot clés simples



Web Sémantique (2000...)



Web + ontologies



Ontologies pour

- Décrire les données publiées sur le web
- Permettre le raisonnement sur ces données
-> recherches enrichies et pertinentes

Applications et Types d'Ontologies

🌐 Ontologies de haut niveau

Décrivent des concepts très généraux et abstraits indépendants d'un domaine spécifique (temps, espace, événement, objet, propriété, relation)

Exemple : DOLCE, BFO, SUMO

📖 Ontologies de domaine

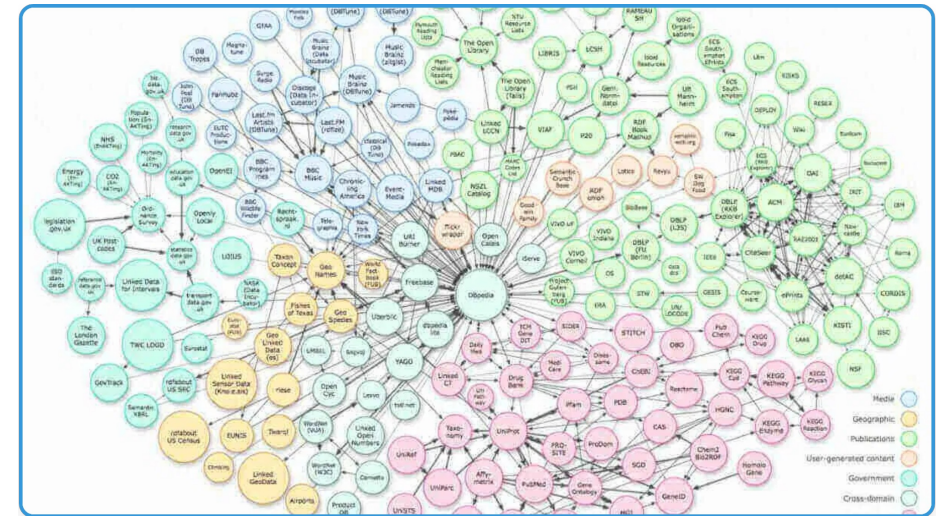
Modélisent un domaine de connaissance particulier (médecine, finance, écologie, droit)

Exemple : SNOMED CT (médecine), Gene Ontology (biologie)

🔧 Ontologies de tâche

Décrivent les concepts et relations liés à une tâche spécifique, indépendamment du domaine

Exemple : Ontologies de diagnostic, de planification



⚙️ Ontologies d'application

Les plus spécifiques, développées pour une application particulière. Combinent des concepts de plusieurs ontologies

Exemple : Ontologie pour un système de recommandation

Un exemple : QBLS (2005)

Un système d'apprentissage basé sur les ontologies

QBLS(Semantic Web Based Learning System)

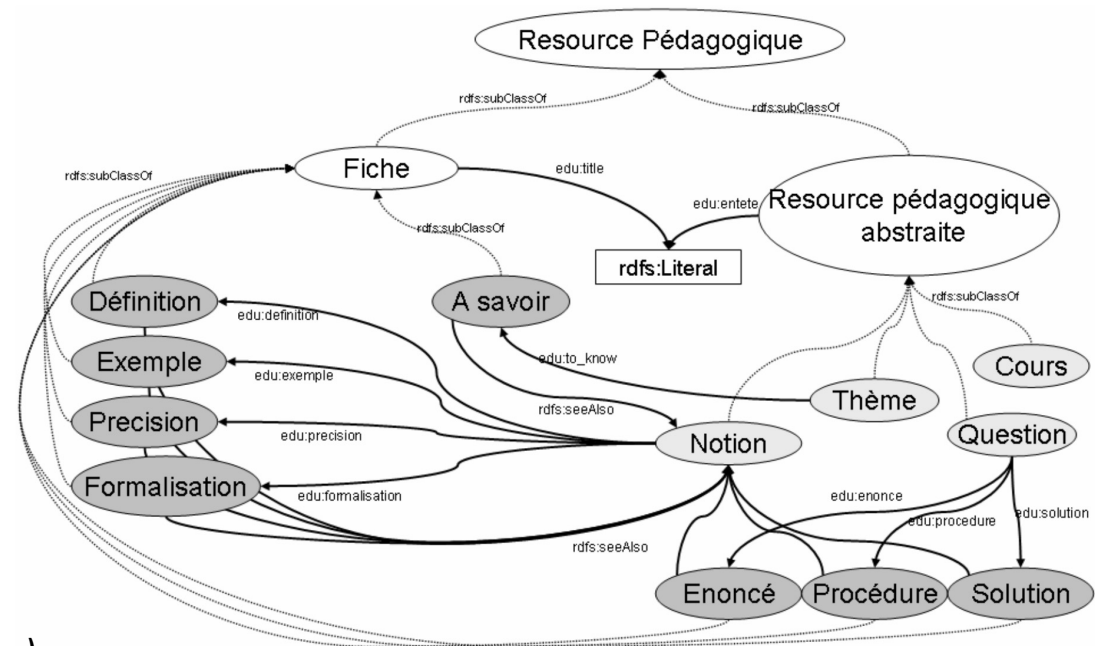
- Raisonnement automatique pour adapter le parcours aux prérequis de l'étudiant en utilisant une ontologie
- Exemple:

Domaine : Programmation en Java

L'ontologie définit :

- les concepts (boucles, classes, héritage...)
- les relations de prérequis (ex comprendre les boucles avant les collections)

L'étudiant qui échoue sur "collections" est redirigé automatiquement vers des ressources sur "boucles"

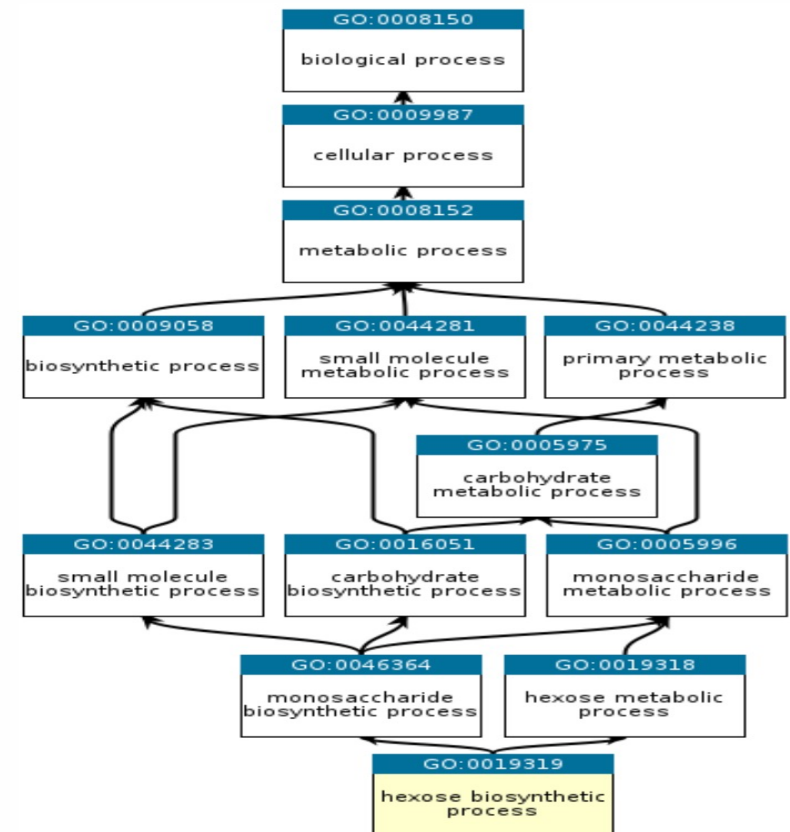


Pour en savoir plus :

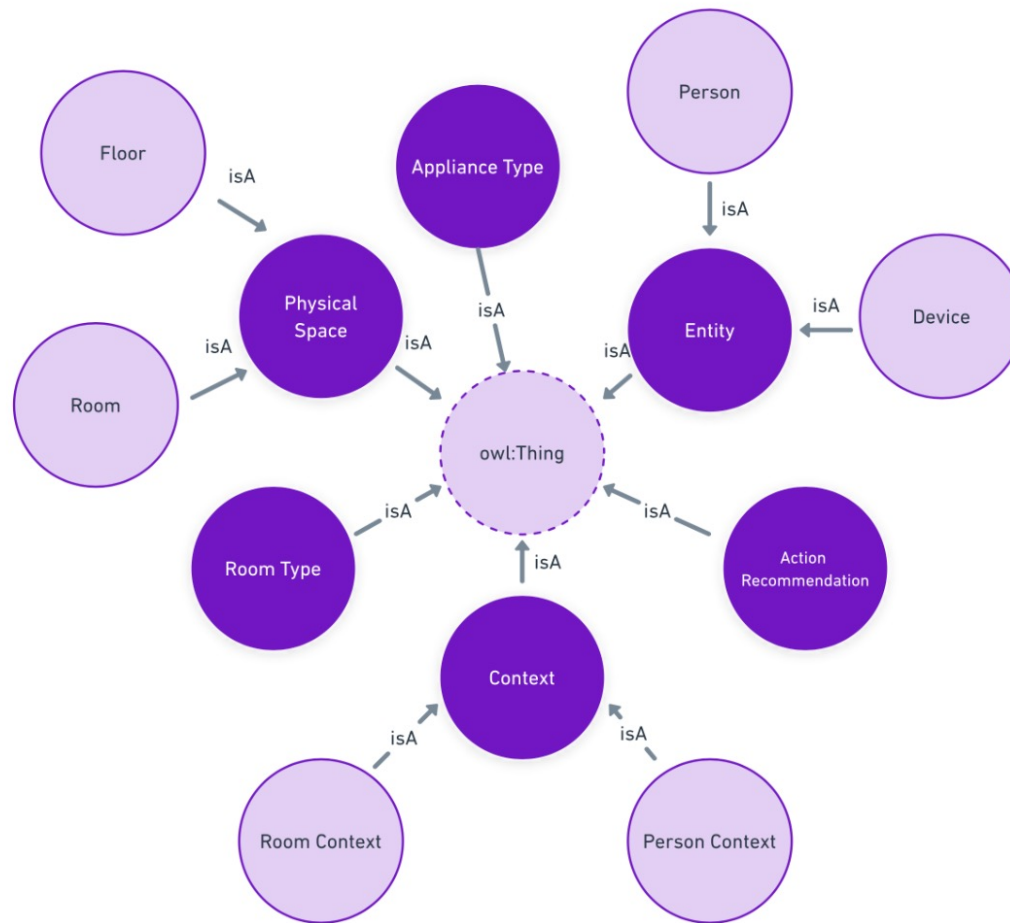
(Dieng & al, 2005) R. Dieng-Kuntz, M. Grandbastien, and D. Hérin, "Web sémantique pour le e-Learning", Actes des journées AFIA / Nice, du 30 mai au 3 juin 2005, http://www-sop.inria.fr/acacia/afia2005/actes/ActesWSeL_Final.pdf

Un exemple en biologie : Gene Ontology

- Représentation structurée et normalisée des études biologiques
- Téléchargeable au format owl (aussi json)



Exemple d'Ontologie pour Smart building

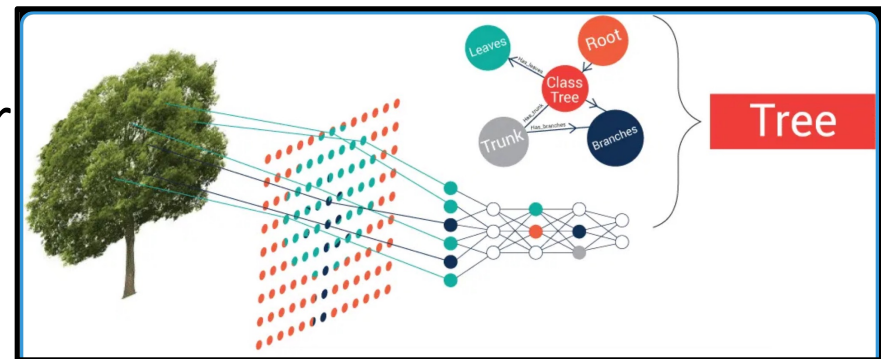


source : Thèse de
doctorat Loris
Plasson, Université de
corse, Décembre
2025

Objectifs des ontologies

■ Partage de connaissances

- Description d'un domaine particulier
- Vocabulaire commun et relations formalisées
- Absence d'ambiguïté



■ Interopérabilité

- Faciliter l'intégration des données provenant de sources hétérogènes

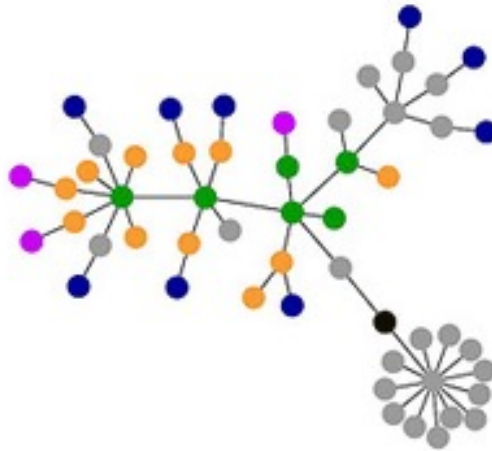
■ Recherche intelligente

- Requêtes basées sur le sens plutôt que sur les mots-clés

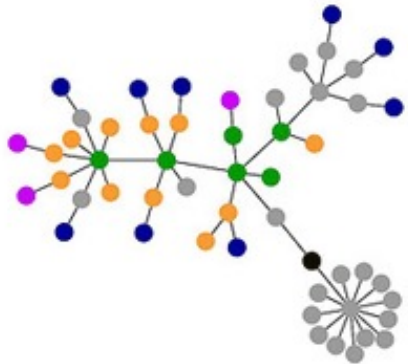
■ Raisonnement automatisé

- Permet d'**inférer de nouvelles connaissances** à partir des connaissances existantes

Langages et standards pour les ontologies



Langages et standards pour les ontologies



Description Logic (DL)

Base formelle logique pour décrire les concepts et relations

2000 Logiques de Description

1999 RDF

Ressource Description Framework

Format de données universel du W3C

2004 OWL

Web Ontology Language

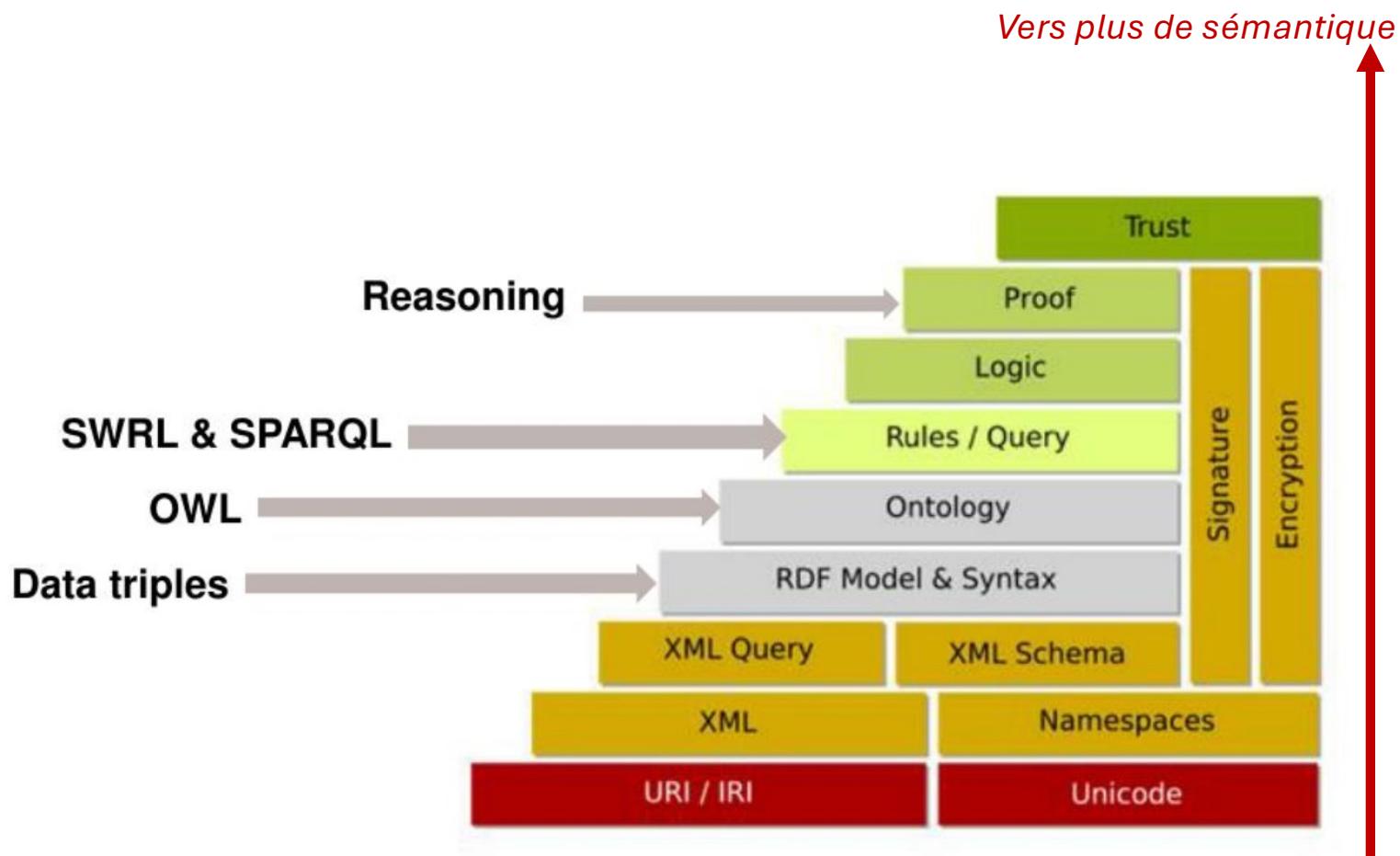
Langage de description basé sur RDF + DL

2008 SPARQL

Sparql Protocol And Tdf Query Language

Langage d'interrogation des graphes RDF/OWL

Un modèle en couche : The Semantic web layer cake



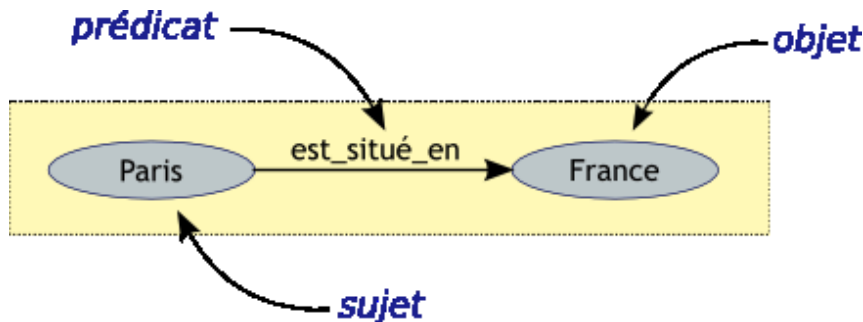
Tim Berners Lee, 2006

RDF : un langage syntaxique pour représenter les données en graphes

RDF (Resource Description Framework) (1999)

- Langage de base du Web sémantique : **modélisation syntaxique**, utile pour l'échange de données
- Permet de représenter des données sous forme de **graphes de triplets** (sujet – prédicat – objet)
- Stocké au RDF/XML

```
<Pays rdf:ID="France"> <nomPays>France</nomPays>
</Pays>
<!-- création de "Paris", qui se trouve en France -->
<Ville rdf:ID="Paris"> <nomVille>Paris</nomVille>
<seTrouveEn rdf:resource="#France" /> </Ville>
```



Simple langage de description

- Peu lisible
- Ne repose pas sur une logique formelle

Turtle (Terse RDF Triple Language)

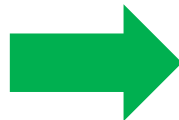
Une syntaxe compacte alternative à RDF : plus lisible

- Utilise des préfixes (@prefix) pour simplifier l'écriture
- Représente directement les triplets (sujet – prédicat – objet)
- Moins verbeux que RDF/XML

```
<rdf:Description
  rdf:about=":alice">
  <rdf:type
    rdf:resource="foaf:Person"/>

  <foaf:firstName>Alice</foaf:fi
    rstName>
</rdf:Description>
```

RDF/XML



```
@prefix foaf: <http://xmlns
com/foaf/0 1/>

:alice a foaf:Person ;
      foaf:firstName "Alice"
```

Turtle

foaf = "friend of a friend"

standardisé W3C en 2014

RDFS : un premier ajout de sémantique à RDF

RDFS (RDF Schema) (1999)

- Extension de RDF pour décrire des vocabulaires (ontologies simples)
- Permet d'introduire de la hiérarchie et des contraintes minimales
 - Classes et sous-classes (rdfs:Class, rdfs:subClassOf)
 - Propriétés et sous-propriétés (rdfs:Property, rdfs:subPropertyOf)
 - Domaine et portée (rdfs:domain, rdfs:range)

Exemple en Turtle

```
:Etudiant a rdfs:Class ;  
          rdfs:subClassOf :Personne .  
  
:enseigneA a rdf:Property ;  
          rdfs:domain :Enseignant ;  
          rdfs:range :Etudiant .
```

Limites

Expressivité faible (pas de
cardinalités, pas de
contraintes complexes)

Les Logiques de description

DL: Description Logics

- Famille de logiques formelles: **fragments** de FOL
- Cadre mathématique pour raisonner sur les concepts, les rôles (relations) et les individus
- Permet de :
 - définir des classes,
 - formuler des contraintes
 - faire de l'inférence automatique (ex : classification, détection d'incohérences)
- DL est indépendant de tout format de fichier ou syntaxe

Principales logiques de description (DL)

DL	Date apparition	Composants clés	Objectif principal	Usage / Base OWL
ALC	1991	$\exists, \forall, \sqcap, \sqcup, \neg$	Base logique expressive	Fondation théorique d'OWL
SHIQ	1999	ALC + hiérarchies, inverses, cardinalités	Représentation riche	OWL 1 DL
SROIQ	2006	SHIQ + nominals, propriétés complexes	Très expressive, encore décidable	OWL 2 DL
EL	2005	$\exists, \sqcap, \top$ (sans négation, ni disjonction)	Raisonnement rapide (polynomial)	OWL 2 EL
DL-Lite	2006	Simple, optimisé pour requêtes	Mapping vers bases de données (RDB)	OWL 2 QL

utilisée dans Protégé 5 6 6

OWL : un langage fondé sur DL pour modéliser des ontologies

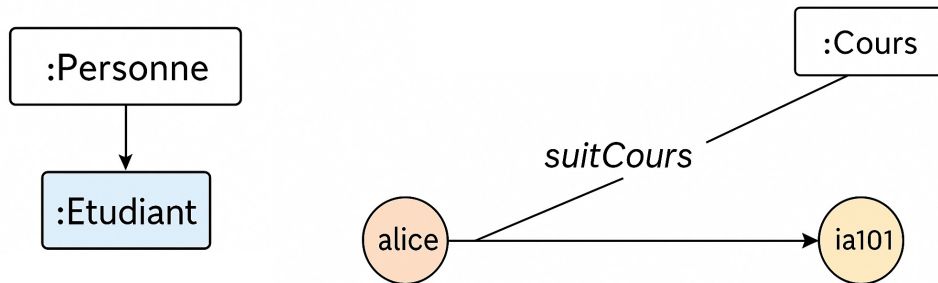


OWL (Web Ontology Language) 

- OWL1 (2004), OWL2 (2012)
- Langage standard du Web sémantique, qui s'appuie explicitement sur certaines DL
- Implémentation concrète d'une DL
- Plusieurs syntaxes : RDF/XML (OWL) ou Turtle/OWL
- Permet d'inférer de nouvelles données avec des reasoners

 **Référence officielle W3C:** <https://www.w3.org/TR/owl2-overview/>

Exemple d'ontologie OWL en en Turtle et RDF/XML



```
@prefix : <http://ex.org/onto#>
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#>
@prefix owl: <http://www.w3.org/2002/07/owl#>
```

```
:Personne a owl:Class
:Etudiant a owl:Class ; rdfs:subClassOf :Personne
:Cours a owl:Class
```

```
:suitCours a owl:ObjectProperty ;
  rdfs:domain :Etudiant ;
  rdfs:range :Cours
```

```
:alice a :Etudiant
:ia101 a :Cours
:alice :suitCours :ia101
```

Turtle / OWL

RDF/XML (OWL)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  xmlns:owl="http://www.w3.org/2002/07/owl#"
  xmlns="http://ex.org/onto#">
  <!-- Classes -->
  <owl:Class rdf:about="http://ex.org/onto#Personne"/>
  <owl:Class rdf:about="http://ex.org/onto#Etudiant">
    <rdfs:subClassOf rdf:resource="http://ex.org/onto#Personne"/>
  </owl:Class>
  <owl:Class rdf:about="http://ex.org/onto#Cours"/>
  <!-- Propriété -->
  <owl:ObjectProperty rdf:about="http://ex.org/onto#suitCours">
    <rdfs:domain rdf:resource="http://ex.org/onto#Etudiant"/>
    <rdfs:range rdf:resource="http://ex.org/onto#Cours"/>
  </owl:ObjectProperty>
  <!-- Individus -->
  <owl:NamedIndividual rdf:about="http://ex.org/onto#alice">
    <rdf:type rdf:resource="http://ex.org/onto#Etudiant"/>
    <suitCours rdf:resource="http://ex.org/onto#ia101"/>
  </owl:NamedIndividual>
  <owl:NamedIndividual rdf:about="http://ex.org/onto#ia101">
    <rdf:type rdf:resource="http://ex.org/onto#Cours"/>
  </owl:NamedIndividual>
</rdf:RDF>
```

OWL ajoute un vocabulaire spécialisé à
RDFS et Turtle (URI prédéfinis)

Comparaison RDF, RDFS et OWL

Besoin	Exemple	RDF	RDFS	OWL
Structuration des données	Max rdf:type Chien (Max est un chien)	✓	✓	✓
Hiérarchie des classes	Chien rdfs:subClassOf Mammifere (Un chien est un mammifère)	✗	✓	✓
Hiérarchie des propriétés	aUneMere rdfs:subPropertyOf aUnParent (Une mère est un parent)	✗	✓	✓
Contraintes de cardinalité	Un mammifère a exactement 2 parents	✗	✗	✓
Définition de classes équivalentes	Voiture $\equiv$ Automobile (Voiture et Automobile sont identiques)	✗	✗	✓
Définition de classes disjointes	Homme $\perp$ Femme (Un individu ne peut pas être les deux à la fois)	✗	✗	✓
Raisonnement automatique avancé	Si Max est un Chien et que Chien $\subseteq$ Mammifere alors on peut dire que Max est aussi un Mammifere	✗	✗	✓

Les **reasoners** : moteurs d'inférence pour OWL

- Exploitent la logique formelle (DL) derrière OWL
- Dérivent automatiquement des faits implicites à partir des axiomes
- Détectent les incohérences, classer les concepts, inférer l'appartenance des individus aux classes

Quelques reasoners	
HermiT	Raisonneur OWL 2, supporte SROIQ, performant
Pellet	Raisonneur complet, supporte SPARQL + règles
Fact++	Très rapide, bon pour les grands ensembles
ELK	Ultra-rapide, mais limité à OWL EL (pas complet)

Langage SPARQL : Principe



- Langage standard W3C pour interroger des données RDF/OWL
- Permet d'interroger des **triplets RDF** : sujet – prédicat – objet
 - phrases sujet-verbe-complément
- Chaque requête SPARQL consiste à **chercher des motifs** qui correspondent à ces triplets
- Syntaxe déclarative : SELECT, WHERE, FILTER, PREFIX
- Permet extraction, filtrage, construction et raisonnement

Hypothèse de
Monde Clos

CWA

CLOSED WORLD ASSUMPTION



- Très utile pour **extraire des données**, y compris les **littéraux** (ex: noms, dates...)
- Mais SPARQL **ne raisonne pas** : il ne trouve que ce qui est **explicitement** dans le graphe RDF

SPARQL Update : modifier des graphes RDF

- Depuis 2013, SparQL permet aussi de modifier des graphes RDF
 - INSERT DATA → ajouter des triplets
 - DELETE DATA → supprimer des triplets
 - DELETE/INSERT ... WHERE → mettre à jour conditionnellement
 - LOAD, CLEAR, CREATE GRAPH, DROP GRAPH → gérer les graphes

```
INSERT DATA {  
    :alice a :Etudiant ;  
           :nom "Alice" .  
}
```

```
DELETE { :alice :nom "Alice" }  
INSERT { :alice :nom "Alice Dupont" }  
WHERE { :alice :nom "Alice" }
```

Langage SWRL (Semantic Web Rule Language)

- Extension à OWL (2004–2005)
- Permet d'exprimer des règles conditionnelles de type si... alors... qu'OWL ne peut pas capturer (ex chaînes de propriétés complexes, contraintes sur des valeurs numériques, etc)
- Syntaxe proche des règles logiques (clauses de Horn) :

```
Person(?x) ^ hasChild(?x, ?y) ^ Doctor(?y) → ParentOfDoctor(?x)
```

Limite
SWRL n'est pas toujours décidable

Outils de Création et d'Édition d'ontologies

- Notre choix : **Protégé**
 - Environnement de développement **open-source**
 - Une version en ligne : WebProtégé



- **TopBraid Composer**



- IDE pour les applications de Web Sémantique

- **NeOn Toolkit**

- Environnement opensource
de développement collaboratif d'ontologies



Construction d'une ontologie avec Protégé



Activité guidée



Plateforme Protégé



 **Développé par Stanford Center**
for Biomedical Informatics Research

 Format et interopérabilité

- Supporte les standards du **Web sémantique** : RDF, RDFS, OWL
- Export possible en RDF/XML, OWL/XML, Turtle...



SOFTWARE

SUPPORT

COMMUNITY

ABOUT

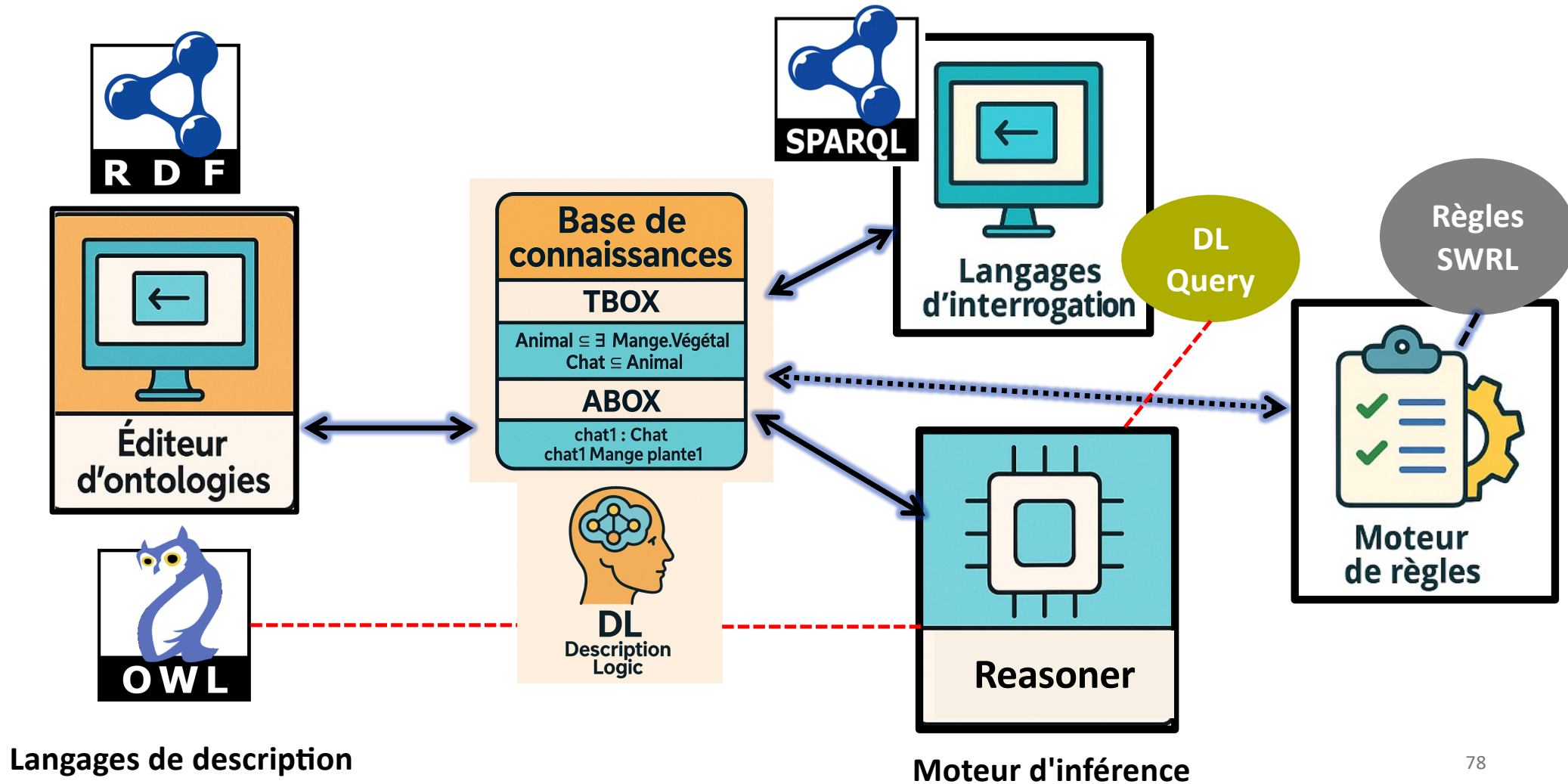
A free, open-source ontology editor
and framework for building
intelligent systems



Avantages

- Interface graphique simple pour découvrir les ontologies
- Très utilisé dans l'enseignement, la recherche et la bio-informatique

Composants de la plateforme Protégé





Installation de Protégé



- Téléchargez la version descTop de Protégé
 - <https://protege.stanford.edu/software.php#desktop-protege>
- Choisissez si possible (non offert pour MacOS) une version incluant le JDK ("Protégé Desktop (with embedded JDK)")
- Téléchargez l'archive zip (tar gz sous Linux)
- **Sous Windows :**
 - Décompressez-la dans un dossier de votre choix (ex C:\Program Files\Protege)
 - Allez dans le dossier et double-cliquez sur run bat
- **Sous macOS :**
 - Décompressez, puis faites glisser **le dossier complet** dans le dossier Applications
 - Dans le dossier, ouvrez run command ou Protégé app
- **Sous Linux :**
 - Allez dans le dossier, ouvrez un terminal
 - Lancez avec : `/run sh`

! Protégé nécessite Java 17+ (LTS)



Si macOS vous empêche d'ouvrir l'app :

Clic droit > Ouvrir (au lieu de double-clic)

Ou allez dans *Préférences Système > Sécurité > Autoriser Protégé*

Création d'une ontologie dans Protégé

Pour créer une nouvelle Ontologie dans protégé :

- File -> new
- IRI = **Uniform Resource Identifier**
- C'est un **identifiant unique et global** pour chaque entité de l'ontologie : classe, propriété, individu...
- Syntaxe : `<namespace>#<nom_local>`

Ex: [http://www univ-corse fr/ontologies/ontocampus#Personne](http://www.univ-corse.fr/ontologies/ontocampus#Personne)

- `http://www univ-corse fr/ontologie` → le **namespace** (préfixe)
- `Personne` → le **nom local** de l'entité
- Le caractère # sert à **séparer** les deux



Contraintes / Bonnes pratiques

- ✓ Le namespace doit être une **URL valide** (même si la ressource n'existe pas en ligne)
- ✓ Il est courant d'utiliser # pour séparer les entités (mais / est aussi autorisé techniquement)
- ⊘ Le nom local **ne doit pas contenir d'espace** ni de caractère spécial
- ✓ Le nom local doit commencer par une **lettre** et peut contenir lettres, chiffres, tirets ou underscores

TP guidé Création d'une ontologie **ontocampus**

Lancez Protégé et créez une nouvelle ontologie

- Lors de la création de l'ontologie, saisissez l'IRI suivante:

IRI = <http://www.univ-corse.fr/ontologies/ontocampus#>

- Choisissez le format de stockage
standard **RDF/XML syntax**
- Saisissez le nom du fichier **ontocampus owl**



Etapes de définition/manipulation d'une ontologie

Définition de l'ontologie (TBOX : vocabulaire)

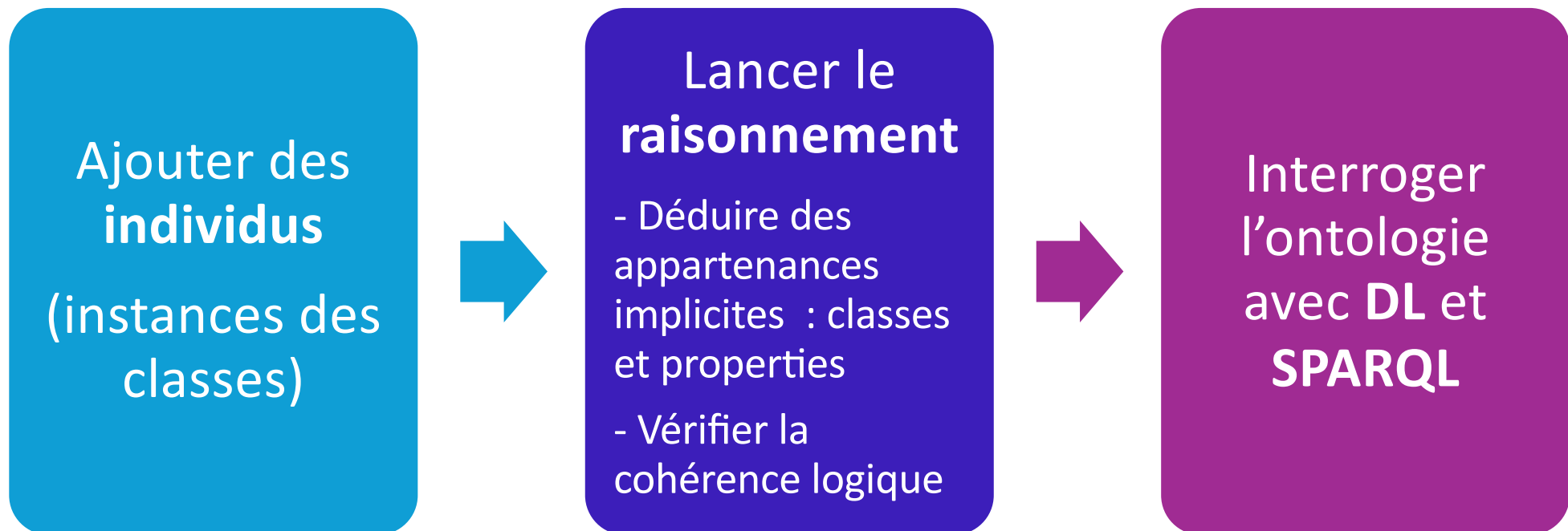
1. Identifier les concepts du domaine → création des **classes primitives (explicites)**
2. Définir les **propriétés**
 - **DataProperty** : valeurs littérales
(ex : *nom*, *adresse*)
 - **ObjectProperty** : liens entre concepts
(ex : *enseigneDans*, *travaillePour*)
3. Spécifier les **contraintes sur les propriétés**
Domaines, portées, cardinalités, inverse, transitivité, etc
4. Créer les **classes complexes (définies par équivalence)**
 - Permettent au raisonneur d'**inférer les instances**
 - Ex : $\text{ÉtudiantTravailleur} \equiv \text{Étudiant} \sqcap (\text{aUnEmploi some Emploi})$



Etapes de définition/manipulation d'une ontologie



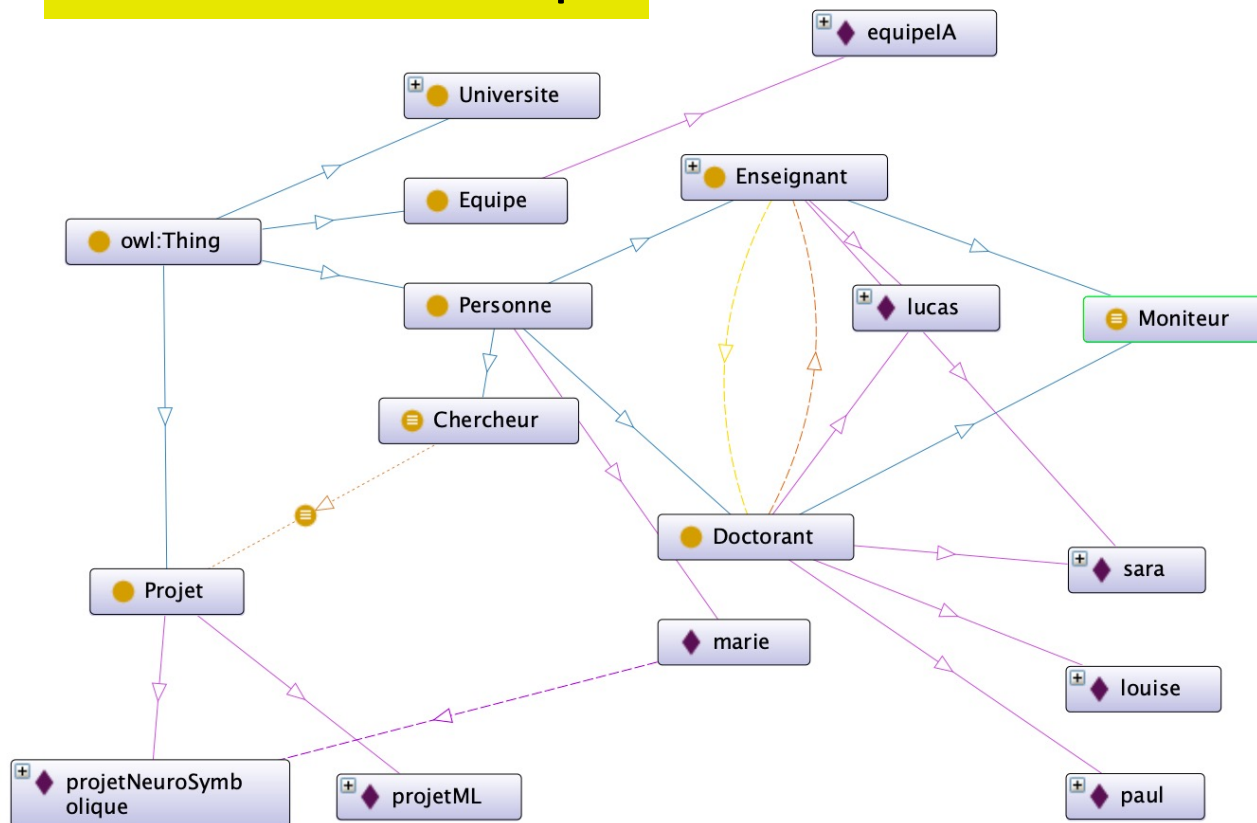
Manipulation et inférence (ABOX)



Définition d'une ontologie avec Protégé



fenêtre OntoGraph

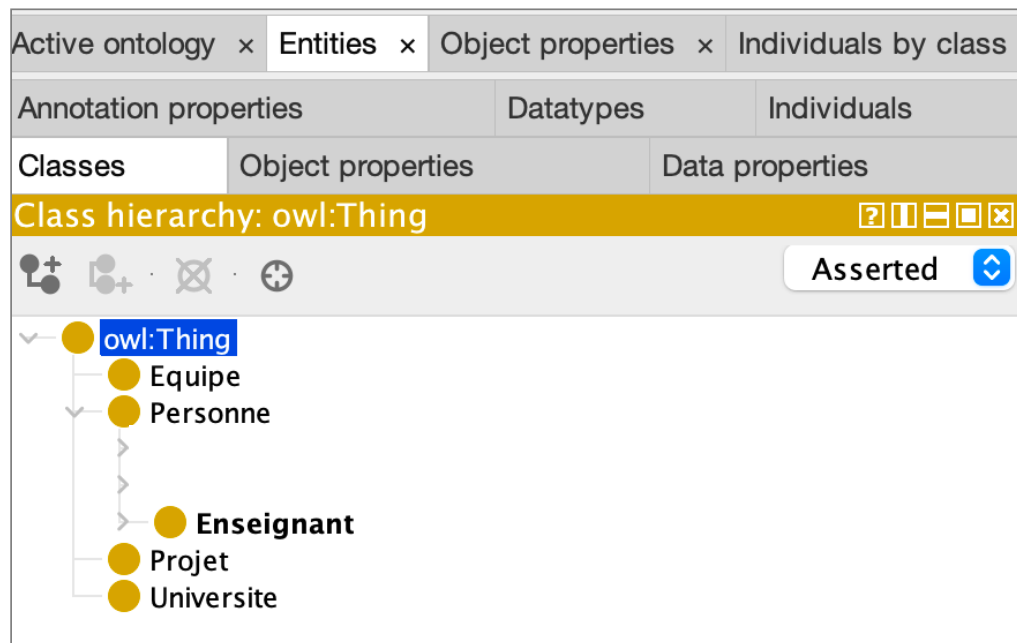


Arc Types	
type filter text	
☒	dirige(Equivalent class some)
☒	encadre (Domain>Range)
☒	encadre(Equivalent class some)
☒	estEncadrePar (Domain>Range)
☒	has individual
☒	has subclass
☒	travailleSur
☒	travailleSur(Equivalent class some)

TP Création d'une ontologie ontocampus

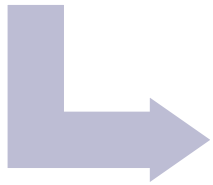


1 Définissez les classes suivantes dans l'onglet Entities puis le sous-onglet Classes



2 – Définition des propriétés

2 1 Définition des **data properties** (attributs)



2 2 - Définition des **Object properties** explicites



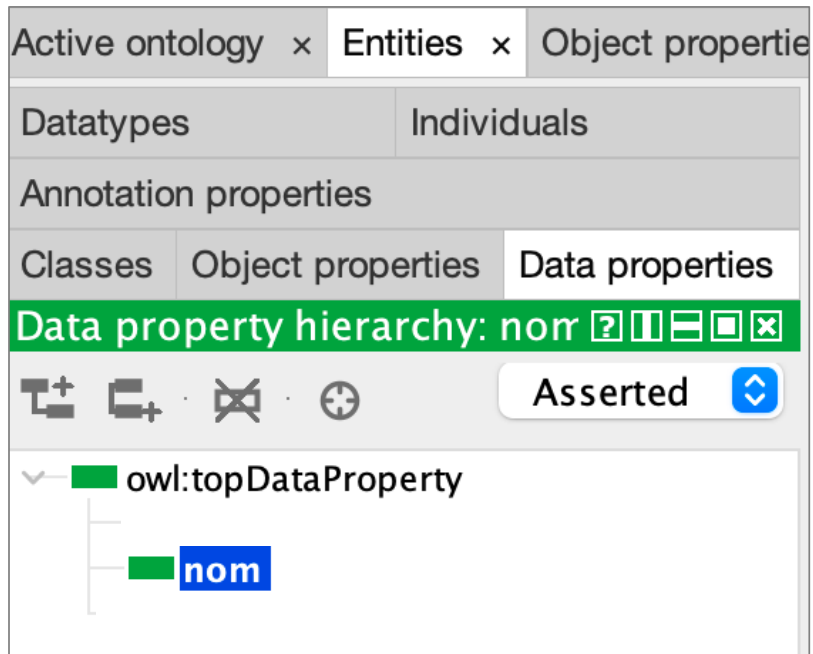
2 3 Définition des propriétés à inférer



2 4 Définition des **Chain of object properties**

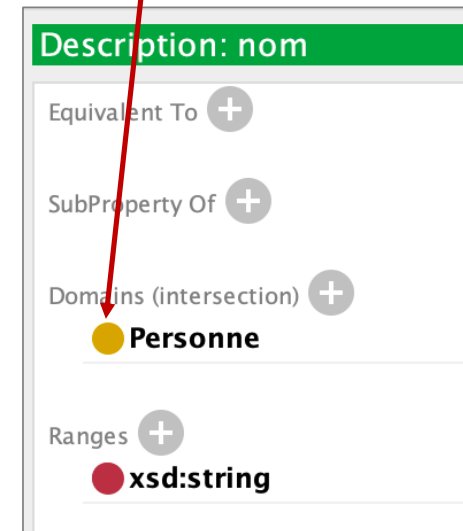
2 1 - Définition des Data properties explicites

Exemple: *nom* dans la classe *Personne*



notion classique d'attribut

Classe que décrit la propriété



2 2 - Définition des Object properties explicites

Exemple:

Un Enseignant encadre un Doctorant

Entite sur laquelle
porte la propriété

Entite cible

Description: encadre

Equivalent To +

SubProperty Of +

Inverse Of +

estEncadrePar

Domains (intersection) +

Enseignant

Ranges (intersection) +

Doctorant

Disjoint With +

SuperProperty Of (Chain) +

Définissent des liens entre classes
(cf association en UML)

Characteristics: affi ? II = □ X

☐ Functional

☐ Inverse functional

☐ Transitive

☐ Symmetric

☐ Asymmetric

☐ Reflexive

☐ Irreflexive

Caractéristiques des Object properties

Caractéristique	Signification	Effet sur l'inférence
Functional (fonctionnelle)	Une entité a au plus un objet lié par cette propriété	Si $x : p y$ et $x : p z$, alors $y = z$
Inverse Functional	Une entité est liée par au plus un sujet	Si $x : p y$ et $z : p y$, alors $x = z$
Symmetric	Si $x : p y$, alors automatiquement $y : p x$	Ex : amiDe est symétrique
Asymmetric	Si $x : p y$, alors jamais $y : p x$	Ex : parentDe (on ne peut pas être parent l'un de l'autre)
Reflexive	Chaque individu est lié à lui-même	$x : p x$ est toujours vrai
Irreflexive	Aucun individu n'est lié à lui-même	$x : p x$ est toujours faux
Transitive	Si $x : p y$ et $y : p z$, alors $x : p z$	Ex : estAncienDe
Inverse Of	Permet de définir une propriété comme l' inverse logique d'une autre	Ex : estEncadréPar est l'inverse de encadre

Characteristics: affi

?

||

=

□

×

☐ Functional

☐ Inverse functional

☐ Transitive

☐ Symmetric

☐ Asymmetric

☐ Reflexive

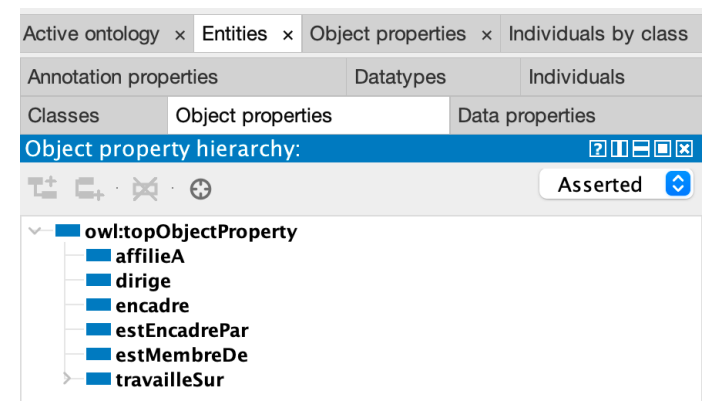
☐ Irreflexive

TP Création d'une ontologie ontocampus

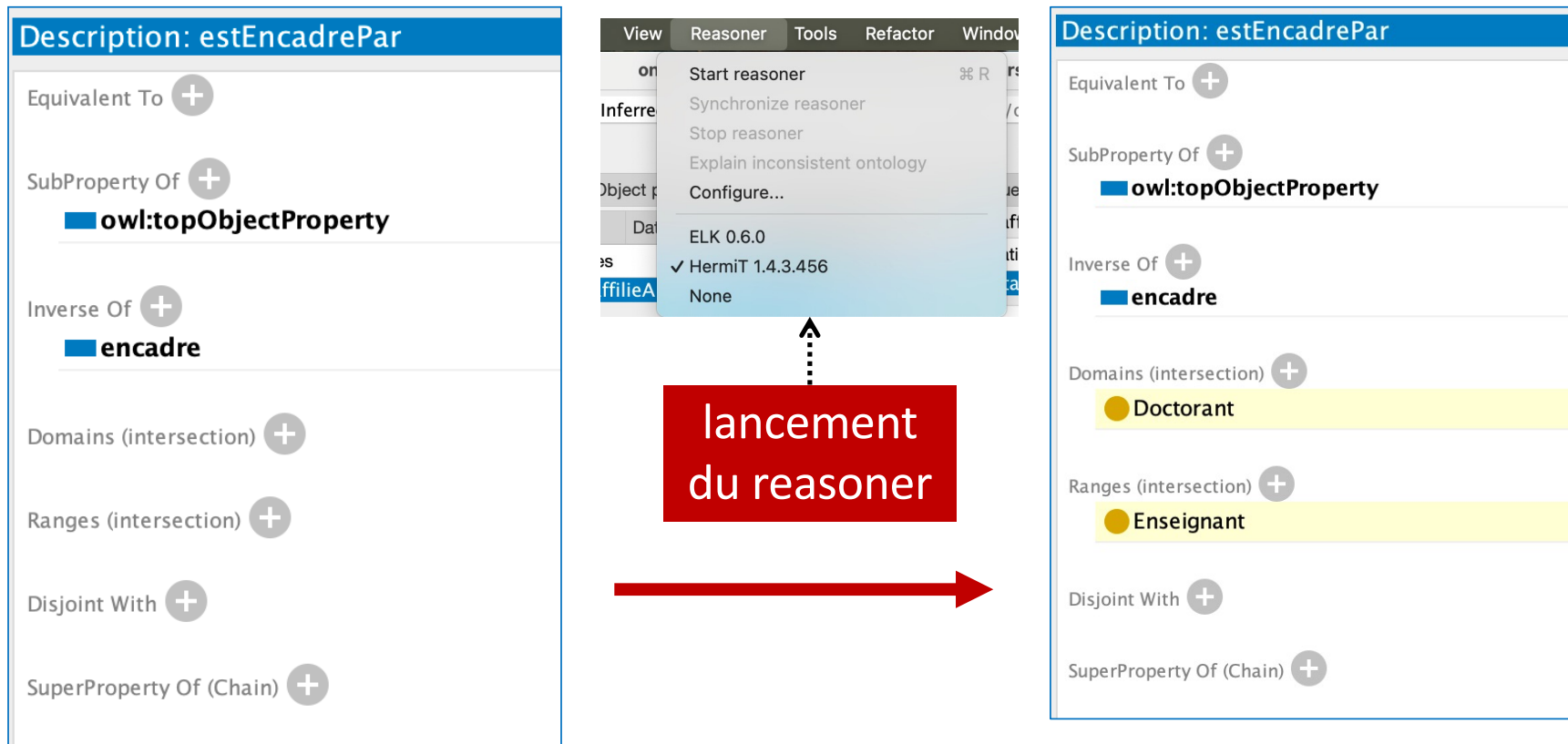


2 1 et 2 2 Définissez les object properties explicites suivantes et ajoutez quelques data properties de votre choix (nom, ect...)

Propriété	Direction	Description
travailleSur	Personne → Projet	Une personne travaille sur un projet de recherche
dirige	Enseignant → Projet	Un enseignant est responsable scientifique d'un projet
encadre	Enseignant → Doctorant	Un enseignant encadre un doctorant
estMembreDe	Personne → Équipe	Une personne est membre d'une équipe de recherche
affilieA	Personne → Université	Une personne est affiliée à une université



2 3 - Définition des Object properties à inférer



TP Création d'une ontologie ontocampus

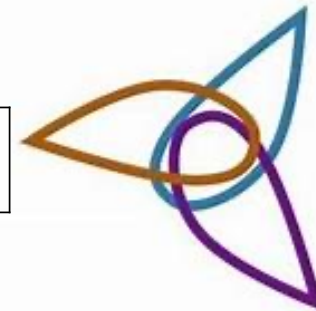


2 3 Définissez l'object propriété à inférer suivante :

- **estEncadrePar**

- inverse de la propriété encadre

estEncadrePar	Doctorant → Enseignant	Un doctorant est encadré par un enseignant (inverse de encadre)
----------------------	------------------------	---



3 – Définition des individus

Active ontology x Entities x Object properties

Datatypes Individuals

Annotation properties

Classes Object properties Data properties

Individuals: alice

- alice
- bob
- equipelA
- jean
- louise
- lucas
- lucie
- marie
- paul
- projetML
- projetNeuroSymbolique
- sara
- univ-corse

Create a new Named individual

Name

IRI

New entity options...

Annuler OK

Description: jean

Types +

Same Individual As +

Different Individuals +

Property assertions: jean

Object property assertions +

Data property assertions +

Negative object property assertions +

Negative data property assertions +

définition de la
classe et des
properties de
l'individu

définition des
propriétés

3 – Définition des individus

Active ontology x Entities x Object properties x Individuals by class x DL Query x SWRLTab x OntoGraf x SPARQL Query x

Datatypes Individuals

Annotation properties

Classes Object properties Data properties

Individuals: alice

alice bob equipelA louise lucas lucie marie paul projetML projetNeuroSymbolique sara univ-corse

alice — <http://www.univ-corse.fr/ontologies/ontocampus#alice>

Annotations Usage

Annotations: alice

Annotations +

Description: alice

Types +

Titulaire

Same Individual As +

Different Individuals +

Property assertions: alice

Object property assertions +

- affilieA univ-corse
- dirige projetNeuroSymbolique
- encadre louise
- encadre lucie
- encadre paul
- estMembreDe equipelA

Data property assertions +

- nom "alice"

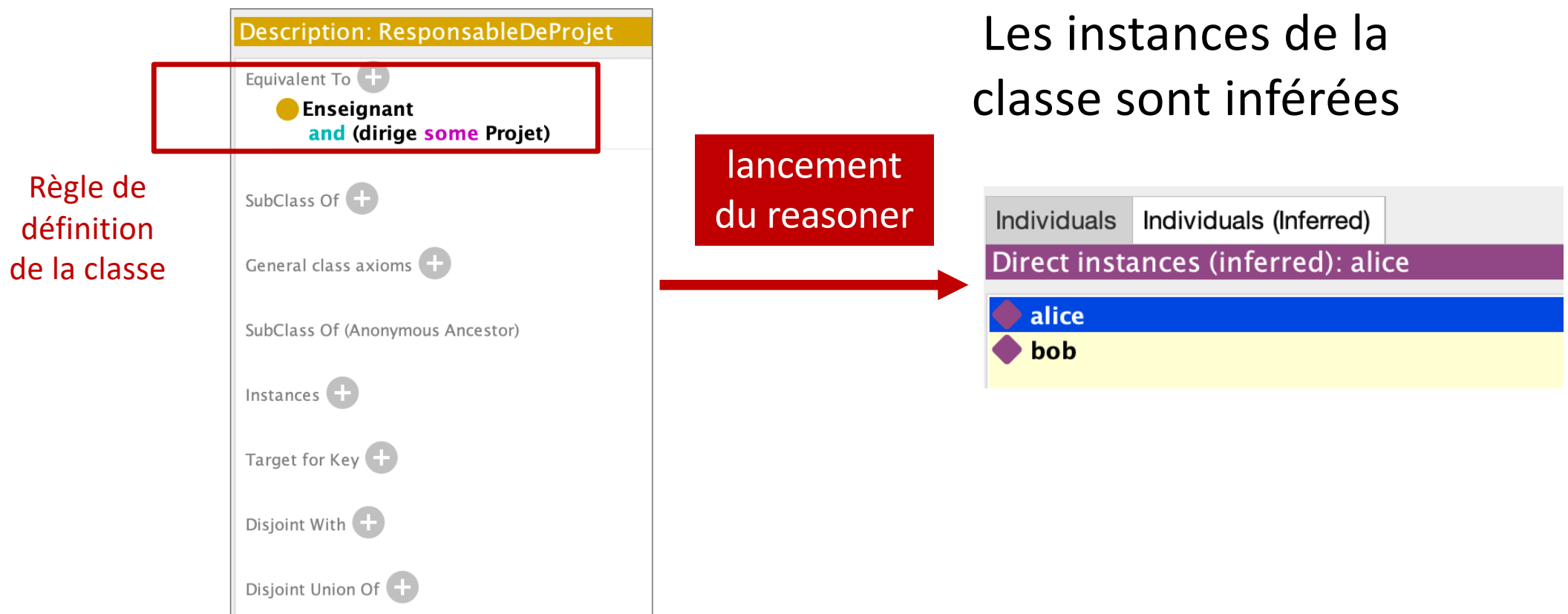
TP Création d'une ontologie ontocampus

3 Définissez les individus suivants :



Individu	Type(s)	Propriétés explicites
alice	Enseignant	affilieA → univ-corse dirige → projetNeuroSymbolique encadre → louise, paul estMembreDe → equipelA
bob	Enseignant	affilieA → univ-corse dirige → projetML encadre → lucas, sara estMembreDe → equipelA
louise	Doctorant	
lucas	Doctorant, Enseignant	
marie	Personne	
paul	Doctorant	travailleSur → projetNeuroSymbolique
sara	Doctorant, Enseignant	
projetML	Projet	
projetNeuroSymbolique	Projet	
equipelA	Équipe	
univ-corse	Université	

Classes complexes = définies par équivalence : les instances sont inférées par le reasoner



Règles permettant l'inférence

- Règles OWL permettant de définir :
 - des classes définies par équivalence : *equivalent to*
 - ➡ classes dont les instances sont inférées selon une règle
 - des axiomes sur les classes: *general class axioms*
 - des contraintes sur les propriétés d'objet et de données
 - ➡ A vérifier lors de l'inférence
- Syntaxe d'écriture des règles : syntaxe lisible Manchester Syntax

OWL Manchester Syntax

- Syntaxe d'écriture des règles : syntaxe lisible

OWL	Manchester	Exemple
someValuesFrom	some	enfant some Homme
allValuesFrom	only	soeur only Femme
hasValue	value	paysDOrigine value angleterre
minCardinality	min	enfant min 3
cardinality	exactly	enfant exactly 3
maxCardinality	max	enfant max 3

lien vers la syntaxe officielle W3C : <https://www.w3.org/2007/OWL/draft/ED-owl2-manchester-syntax-20081128/>

Syntaxe OWL / DL query

Concept logique	Notation mathématique	Syntaxe Protégé (DL Query)	Exemple
Intersection	$A \sqcap B$	A and B	Personne and Enseignant
Union	$A \sqcup B$	A or B	Doctorant or Enseignant
Négation	$\neg A$	not A	not Doctorant
Inclusion	$A \sqsubseteq B$	— (via par "sous-classes")	Doctorant $\sqsubseteq$ Personne (axiome)
Équivalence	$A \equiv B$	— (via « Equivalent classes »)	Enseignant $\equiv$ Personne and (encadre some Doctorant)
Existence	$\exists R C$	R some C	encadre some Doctorant
Universalité	$\forall R C$	R only C	encadre only Doctorant
Valeur exacte	$R = a$	R value a	encadre value paul
Cardinalité exacte	$= n R C$	R exactly n C	encadre exactly 2 Doctorant
Cardinalité minimale	$\geq n R C$	R min n C	encadre min 1 Doctorant
Cardinalité maximale	$\leq n R C$	R max n C	encadre max 3 Doctorant

TP Création d'une ontologie ontocampus



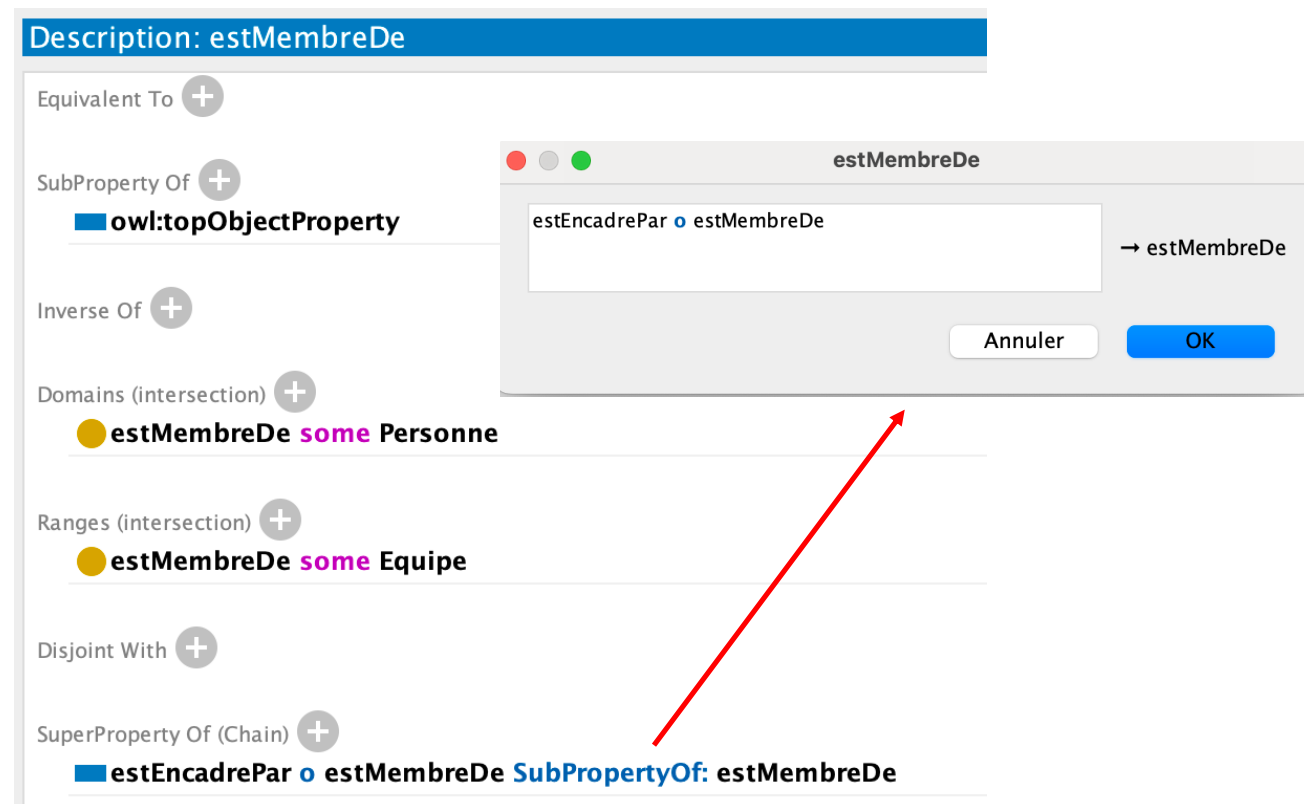
5 Définissez les Classes suivantes définies par équivalence :

- la classe **ResponsableDeProjet** hérite de Titulaire définie par la règle
 - Est un Enseignant qui dirige un projet cf exemple page 88
- la classe **Chercheur** hérite de Personne définie par la règle
 - Est une Personne qui travailleSur un projet
- la classe **Moniteur** hérite de Doctorant et Enseignant définie par la règle
 - Est un Doctorant et un Enseignant
- la classe **Encadrant** hérite de Enseignant et définie par la règle : Est Enseignant et encadre un doctorant
- la classe **EncadrantExperimente** : Encadrant qui encadre au minimum 3 doctorant



Définition de Chain of object properties (SuperProperty of chain)

- Le symbole **o** correspond à la composition
 - similaire à l'opérateur de composition de fonctions
- Signifie :
Si x estEncadrePar y et
 y estMembreDe z ,
alors on en déduit que
 x estMembreDe z



La valeur de la propriété sera inférée par le reasoner

TP Création d'une ontologie ontocampus



6 Définissez les **superProperties of (Chain)** suivantes :



exemple page
précédente

- Complétez la propriété estMembreDe par la chaine :
 - Si une Personne p est encadrée par X et que X estMembreDe l'équipe E Alors p estMembre de l'équipe E
- Complétez la propriété travailleSur par la chaine :
 - Si une Personne p dirige un projet x alors p travaille sur ce projet x
- Complétez la propriété affiliéA par la chaine :
 - Si une Personne p est encadrée par X et que X est affiliéA université U Alors p est affiliéA à université U

TP Création d'une ontologie ontocampus



7 Test des inférences

- Lancer le **reasoner (hermiT)**
- Visualiser les individus et les propriétés inférés



Apparaissent en jaune
après lancement du
reasoner

Active ontology x Entities x Object properties x Individuals by class x

Class hierarchy: Moniteur

owl:Thing
Equipe
Personne
Chercheur
Docteurant
Moniteur
Enseignant
Projet
Universite

Individuals Individuals (Inferred)

Direct instances (inferred):

lucas
sara

Active ontology x Entities x Object properties x Individuals by class x OWL Viz x DL Query x OntoGraf x SPARQL Query x

Annotation properties x Datatypes x Individuals

Classes Object properties Data properties

Individuals: alice

alice
bob
equipelA
louise
lucas
marie
paul
projetML
projetNeuroSymbolique
sara
univ-corse

Annotations Usage

Annotations: alice

Annotations

Description: alice

Types

Enseignant
Encadrant
ResponsableDeProjet

Same Individual As

Different Individuals

Property assertions: alice

Object property assertions

affilieA univ-corse
dirige projetNeuroSymbolique
encadre louise
encadre paul
estMembreDe equipelA
travailleSur projetNeuroSymbolique

Data property assertions

Negative object property assertions

Negative data property assertions

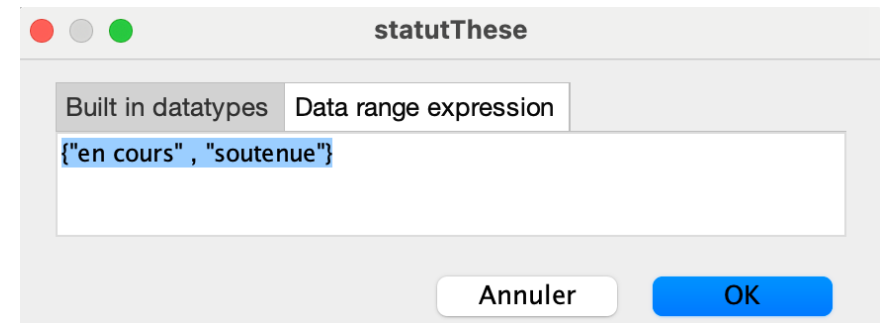
103

TP Création d'une ontologie ontocampus



8 1 Quand les Doctorants deviennent Docteurs !

- Ajoutez une data propriété "**statutThese**" de type String dans la classe Doctorant
- Définissez une expression
range {"en cours", "soutenue"}
- Dans la classe Doctorant,
 - Ajoutez la règle suivante dans l'onglet subclass OF
statutThese only {"en cours", "soutenue"}



TP Création d'une ontologie ontocampus



8 2

- Ajoutez une classe Docteur qui hérite de Doctorant avec une règle de définition par équivalence :
 - *Doctorant and (statutThese value "soutenue")*
- Définissez la propriété statutThese pour tous les individus doctorants à la valeur "en cours" sauf Lucas qui a déjà thèse: son statutThese="soutenue"
- Lancez le reasoner et vérifiez que lucas apparait bien comme une instance de Docteur

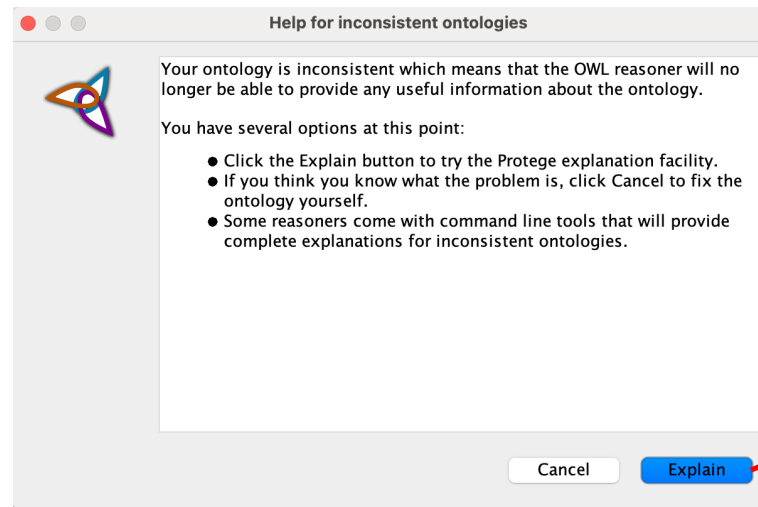


TP Création d'une ontologie ontocampus



8 3

- Arrêtez le reasoner
- Modifiez le statutThese de lucie en introduisant volontairement une erreur : statutThese="enCours" (et non "en cours")
- Sauvegardez
- Relancez le reasoner
 - Il devrait détecter le problème



Visualisez
l'explication de
l'erreur

Interroger une Ontologie dans Protégé

Description Logic DL Query

DL query:

Query (class expression)

Enseignant and (encadre value paul)

Execute Add to ontology

Query results

Instances (1 of 1)

- alice

DL query:

Query (class expression)

Enseignant

Execute Add to ontology

Query results

Instances (4 of 4)

- alice
- bob
- lucas
- sara

Query for

- ☐ Direct superclasses
- ☐ Superclasses
- ☐ Equivalent classes
- ☐ Direct subclasses
- ☐ Subclasses
- ☒ Instances

SPARQL

SPARQL query:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

PREFIX : <http://www.univ-corse.fr/ontologies/ontocampus#>
SELECT ?enseignant
WHERE {
  ?enseignant rdf:type :Titulaire .
  ?enseignant :encadre :paul .
}
```

enseignant
alice

Execute

DL Query dans Protégé

Objectif : Interroger la base de connaissances à l'aide de la logique des descriptions

 **Syntaxe inspirée de la logique de description :**

- Enseignant and (encadre **some** Doctorant)
→ Tous les enseignants qui encadrent **au moins un doctorant**
- Personne and not (encadre **some** Projet)
→ Personnes **n'encadrant aucun projet**
- Projet and (dirige **only** Professeur)
→ Projets dirigés **uniquement** par des professeurs



Attention

- Fonctionne sous l'**hypothèse du monde ouvert (OWA)**
- Utilise le **raisonneur actif** (HermiT, Pellet, etc)

TP Création d'une ontologie ontocampus



9 Interrogez la base de connaissance avec DL Query

Exprimez les requêtes suivantes :

- a) Lister tous les noms des doctorants
- b) Lister les personnes affiliées à l'université de corse
- c) Lister les doctorants qui participent à un projet
- d) Lister les enseignants qui sont aussi encadrants
- e) Les projets dirigés par bob
- f) Les doctorants encadrés par bob



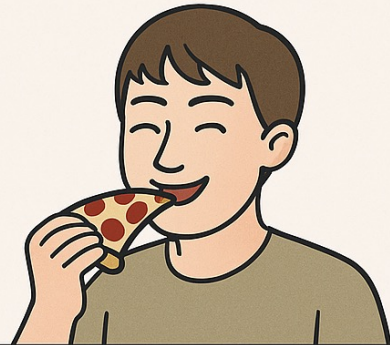
Interprétation des informations manquantes

Hypothèse de Monde Ouvert

Hypothèse de Monde Clos

OWA

OPEN WORLD ASSUMPTION



A man eats pizza

Assumed to be true

CWA

CLOSED WORLD ASSUMPTION



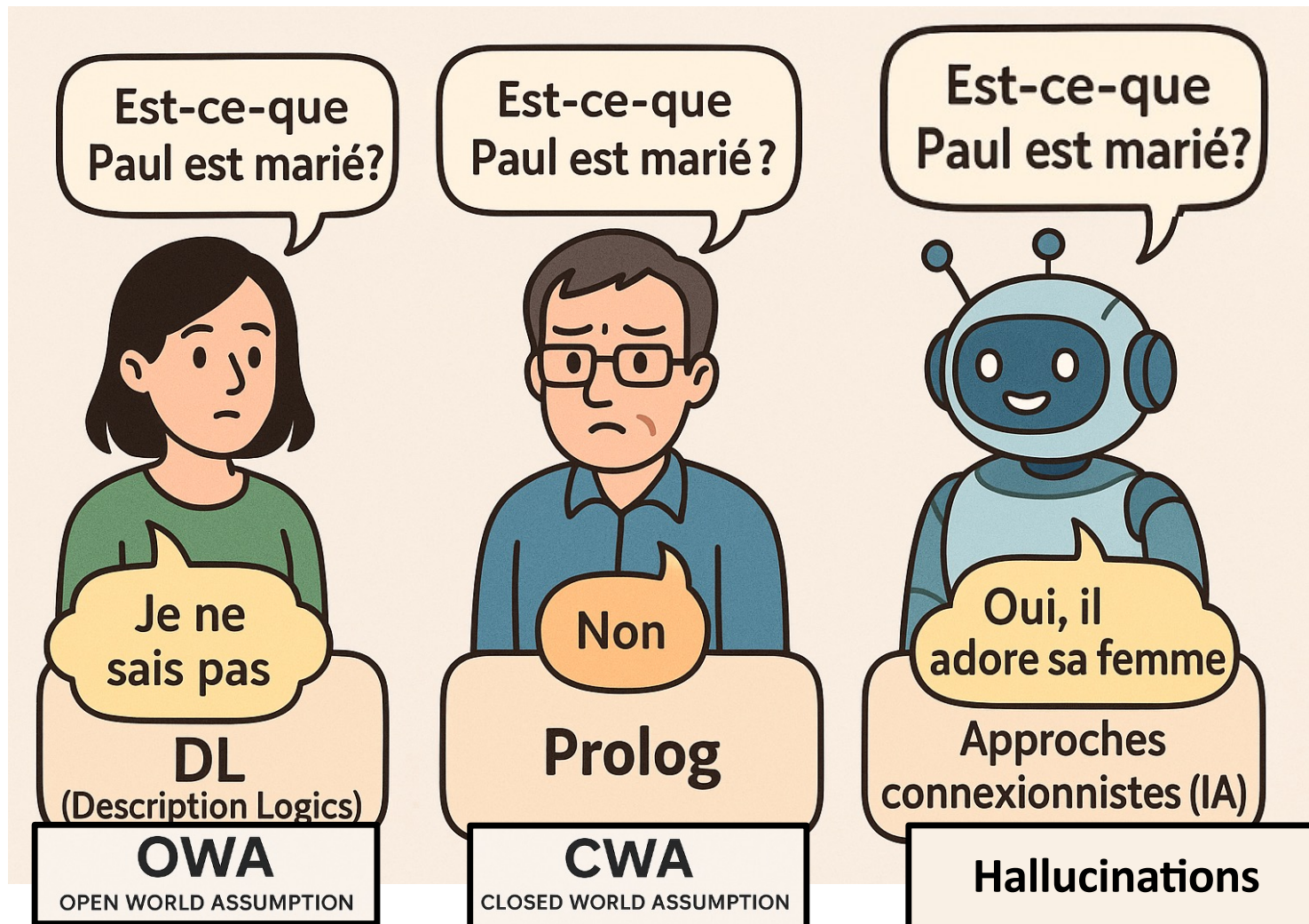
A man ats pizza

Unknown

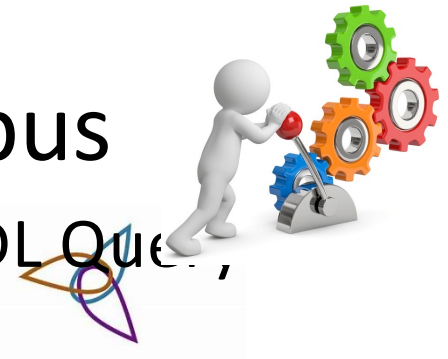
- Description Logics
- Ontologies

- **SparQL**
- Systèmes Experts
- Bases de données

3 approches d'Interprétation des informations manquantes



TP Création d'une ontologie ontocampus



9(suite) Interrogez la base de connaissance avec DL Query,

Exprimez la requete suivante :

g) Les encadrants qui n'encadrent pas lucas

Que remarquez vous?

Quel est l'influence de l'Hypothèse de Monde Ouvert (OWA)?



Langage SparQL : Principe



- Langage standard W3C pour interroger des données RDF/OWL
- Permet d'interroger des **triplets RDF** : sujet – prédicat – objet
 - phrases sujet-verbe-complément
- Chaque requête SPARQL consiste à **chercher des motifs** qui correspondent à ces triplets
- Syntaxe déclarative : SELECT, WHERE, FILTER, PREFIX
- Permet extraction, filtrage, construction et raisonnement

Hypothèse de
Monde Clos

CWA

CLOSED WORLD ASSUMPTION



- Très utile pour **extraire des données**, y compris les **littéraux** (ex: noms, dates...)
- Mais SPARQL **ne raisonne pas** : il ne trouve que ce qui est **explicitement** dans le graphe RDF

Structure d'une requête SPARQL



```
PREFIX : <http://www.univ-corse.fr/ontologies/ontocampus#>
```

IRI de l'ontologie définie lors de sa création

```
SELECT ?x ?y
```

```
WHERE {
```

```
  ?x :aPourNom ?y
```

```
}
```

sujet prédicat objet

PREFIX

Définit un alias (: ici) pour éviter de répéter l'URI

SELECT

Spécifie les **variables** à retourner (commençant par ?)

WHERE {}

Contient le **modèle de triplets** à faire correspondre

Requête SPARQL simple (1)



🎓 Trouver tous les individus de type Personne

```
PREFIX : <http://www.univ-corse.fr/ontologies/ontocampus#>
```

```
SELECT ?x
```

```
WHERE {
```

```
    ?x rdf:type :Personne
```

```
}
```

sujet	prédictat	objet
-------	-----------	-------

Requête SPARQL simple (2)



🎓 👤 Trouver toutes les titulaires qui encadrent un doctorant

PREFIX : `<http://www.univ-corse.fr/ontologies/ontocampus#>`

```
SELECT distinct ?pers
WHERE {
    ?pers rdf:type :Titulaire
    ?pers :encadre ?etudiant
    ?etudiant rdf:type :Doctorant
}
```

ET logique entre les motifs
tous les motifs doivent être
vrais simultanément

Langage SparQL : Opérateurs logiques

Opérateur	Signification	Exemple
OPTIONAL	Clause optionnelle	OPTIONAL { ?x :aPourAge ?age }
UNION	Disjonction (OU)	{ } UNION { }
FILTER	Filtrage (SI)	FILTER(?age > 18)
BIND	Création de variable	BIND(?age * 2 AS ?doubleAge)

DLQuery VS Sparql : résumé

	 DL Query	 SPARQL
Raisonnement	✓ Oui (via reasoner)	✗ Non (par défaut)
Syntaxe	Logique déclarative (conceptuelle)	Triplets RDF + filtres SPARQL
Orientation	Sémantique : concepts et inférences	Structurelle : graphe RDF explicite
Extraction de littéraux	✗ Non	✓ Oui (FILTER, OPTIONAL, BIND...)
Expressivité logique	Limitée aux concepts OWL/DL	Très souple, mais sans logique inférentielle
Usage typique	Tester définitions, requêtes conceptuelles	Exporter, filtrer, manipuler des données

TP Création d'une ontologie ontocampus



10 Interrogez la base de connaissance avec SPARQL

■ Exprimez les requêtes précédentes en SPARQL

- a) Lister tous les noms des doctorants
- b) Lister les titulaires affiliées à l'université de corse
- c) Lister les doctorants qui participent à un projet
- d) Lister les titulaires qui sont aussi encadrants
- e) Les projets dirigés par bob
- f) Les doctorants encadrés par bob
- g) Les doctorants qui n'ont pas encore soutenu
- h) Les titulaires qui n'encadrent pas lucas



**Hypothèse de
Monde Clos**

CWA

CLOSED WORLD ASSUMPTION



Plusieurs requêtes n'ont aucun résultat : expliquez pourquoi ?

■ Que constatez-vous au niveau des données inférées ?

Pour aller plus loin dans les règles :

le langage SWRL (Semantic Web Rule Language)

- Objectif SWRL : Définir des règles plus expressives
- Inférences complexes impliquant plusieurs individus ou des conditions combinées qui ne peuvent pas être exprimées en OWL seul :
 - Contraintes **temporelles** (currentDateTime, comparaisons),
 - Patrons **n-aires** (rôles, statuts, quantités attachées à une relation),
 - Heuristiques « au moins N » via variables distinctes,
 - Vérifications **éthiques/organisationnelles** (conflits, double rôle, etc)



Interopérabilité et raisonnement :

- Compatible avec **Drools**, **Jess**, ou moteurs de règles embarqués
- Export possible via l'onglet **SWRLTab** dans Protégé
(OWL+SWRL → moteur → OWL enrichi)

Règles SWRL : Un Exemple

- Deux doctorants sont dits **colleguesDoctorant** s'ils sont encadrés par le même encadrant
- Impossible à exprimer en OWL :
 - relation entre deux individus en fonction d'un autre individu lié à chacun
 - logique triangulaire (si d1 et d2 partagent p)

```
Doctorant(?d1) ^  
Doctorant(?d2) ^  
Enseignant(?e) ^  
encadre(?e, ?d1) ^  
encadre(?e, ?d2) ^  
differentFrom(?d1, ?d2) ->  
aPourCollegueDoctorant(?d1,  
?d2)
```



CWA

CLOSED WORLD ASSUMPTION

Exécuté par un moteur de règles (similaire au moteur d'inférence des systèmes experts)

Exemple dans Protégé

The screenshot shows the Protégé SWRLTab interface. At the top, there are tabs for 'Active ontology', 'Entities', 'Object properties', 'Individuals by class', 'DL Query', 'SWRLTab', and 'SPARQL Query'. The 'SWRLTab' is active, displaying a table with columns 'Name', 'Rule', and 'Comment'. A single rule is listed with the name 'colleaguesDoctorants' and the rule text 'ontocampus:Doctorant(?d1) ^ ontocampus:Doctorant(?d2) ^ ontocampus:Enseignant(?e) ^ ontoc...'. Below the table are buttons for 'New', 'Edit', 'Clone', and 'Delete'. At the bottom, there is a 'Control' panel with tabs for 'Rules', 'Asserted Axioms', 'Inferred Axioms', and 'OWL 2 RL'. The 'Rules' tab is selected, showing instructions on using the Drools rule engine. At the bottom of the 'Rules' tab are three buttons: 'OWL+SWRL->Drools', 'Run Drools', and 'Drools->OWL'. Red dotted lines connect these buttons to the numbered steps in the diagram below.

Name	Rule	Comment
☒ colleaguesDoctorants	ontocampus:Doctorant(?d1) ^ ontocampus:Doctorant(?d2) ^ ontocampus:Enseignant(?e) ^ ontoc...	

Control Rules Asserted Axioms Inferred Axioms OWL 2 RL

Using the Drools rule engine.

Press the 'OWL+SWRL->Drools' button to transfer SWRL rules and relevant OWL knowledge to the rule engine.
Press the 'Run Drools' button to run the rule engine.
Press the 'Drools->OWL' button to transfer the inferred rule engine knowledge to OWL knowledge.

The SWRLAPI supports an OWL profile called OWL 2 RL and uses an OWL 2 RL-based reasoner to perform reasoning.
See the 'OWL 2 RL' sub-tab for more information on this reasoner.

OWL+SWRL->Drools Run Drools Drools->OWL

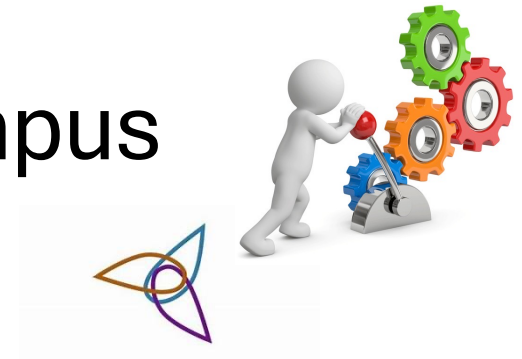
1 – Chargement de l'ontologie et des règles dans le moteur Drools

2 - Exécution des règles

3- Transfert des faits inférés dans l'ontologieOWL

TP Création d'une ontologie ontocampus

11



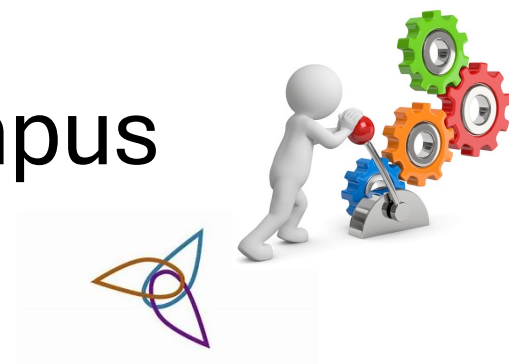
- Ajoutez l'object propriete "*aPourCollegueDoctorant*"
- Définissez la règle SWRL de la page 113
 $\text{Doctorant}(?d1) \wedge \text{Doctorant}(?d2) \wedge \text{Enseignant}(?e) \wedge$
 $\text{encadre}(?e, ?d1) \wedge \text{encadre}(?e, ?d2) \wedge \textbf{differentFrom}(?d1, ?d2) \rightarrow$
 $\text{aPourCollegueDoctorant}(?d1, ?d2)$
- Visualisez les individus en prenant soin d'arreter le reasoner
- Que constatez-vous?

TP Création d'une ontologie ontocampus

11(suite)

- Créez une classe EnseignantSansDoctorant
- Ajoutez un individu Jean dans la classe Titulaire
- Définissez la règle SWRL suivante

```
Enseignant(?e) ^ not(encadre(?e, ?d) )  
→ EnseignantSansDoctorant(?e)
```



Comment exploiter une ontologie ?

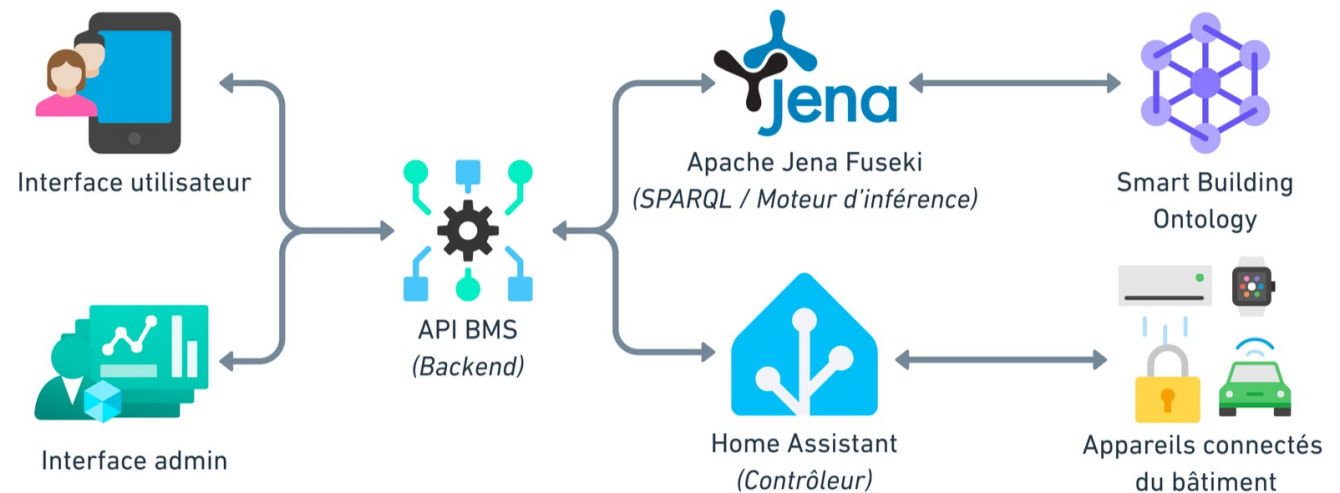
Un exemple pour introduire des pistes



Building Management System

Exemple d'Exploitation opérationnelle d'une ontologie

- Exportation de l'ontologie vers un serveur Jena/Fuseki
- L'ontologie est interrogeable et modifiable dynamiquement via Jena/Fuseki en SPARQL

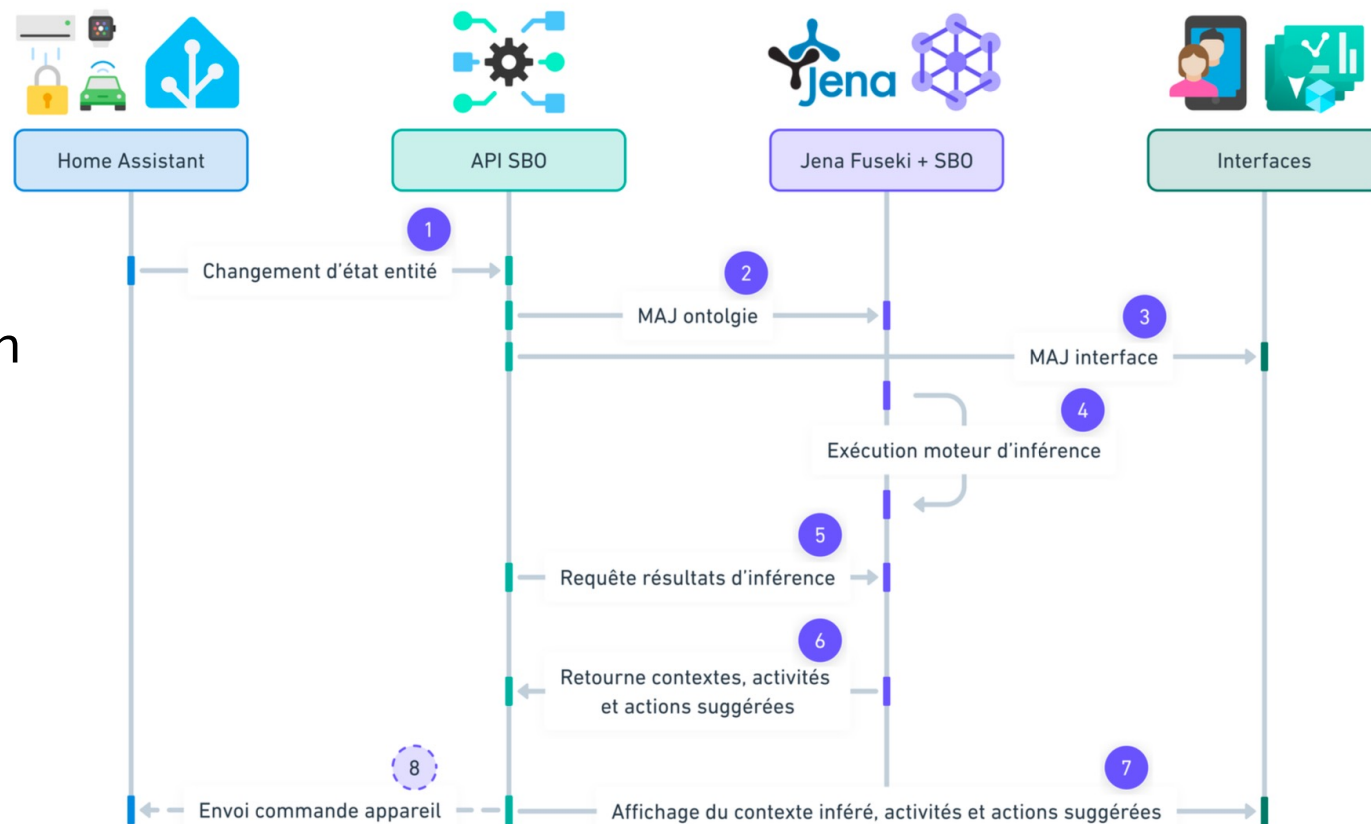


Architecture du Building Management System, avec l'API SBO au cœur du dispositif et assurant la communication entre les différents acteurs

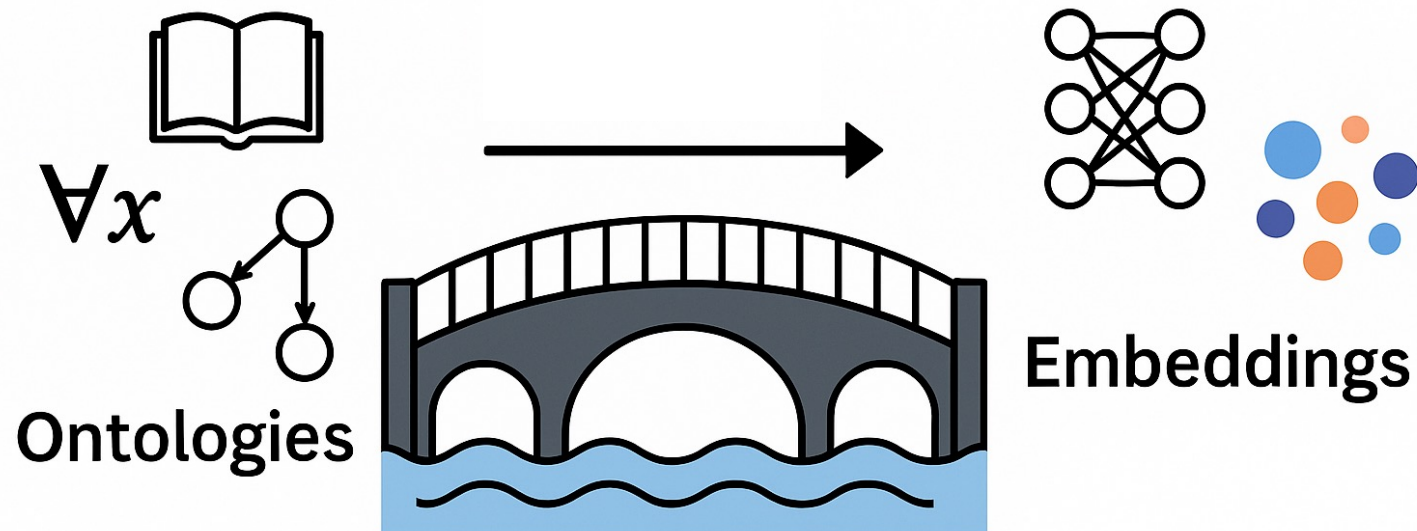
Building Management System

Exemple d'Exploitation opérationnelle d'une ontologie

Scénario d'utilisation
de l'API SBO
Management pour
modifier l'ontologie



Pour conclure avant le connexionnisme et les embeddings ...



Perspectives : Couplage LLM - Ontologie

🧠 + 🧩 Dans un RAG enrichi par une ontologie

L'ontologie permet :

- la structuration explicite du savoir (types, relations, contraintes) ;
- la navigation sémantique raisonnée (via SPARQL, raisonneurs, règles SWRL, etc)

Le LLM peut être guidé ou corrigé par la structure et les inférences fournies par l'ontologie



Approche IA **neuro-symbolique**

L'ontologie,
un allié qui
comprend



Apports concrets des ontologies au RAG

RAG sémantique (Semantic Retrieval-Augmented Generation)

- Extraire dynamiquement des **triplés RDF**, définitions ou fragments d'ontologie
- Les injecter dans le **contexte du prompt**
- 👉 Exemple : “Voici la définition formelle d'un *Encadrant* À partir de cela, explique si Alice peut être qualifiée ainsi ”

Génération de requêtes

- LLM → génère des requêtes SPARQL ou SWRL à partir de questions naturelles

Contrôle des hallucinations

- Vérifier les faits énoncés par le LLM via l'ontologie
- Appliquer des contraintes ou des rejets

Explication interactive

Le LLM peut naviguer dans l'ontologie pour répondre à des questions d'apprentissage

Il peut s'aider des *labels, commentaires, relations hiérarchiques*, etc

Création ou extension d'ontologie

Le LLM peut proposer de nouvelles classes ou propriétés à partir d'un corpus (texte, documents), mais l'utilisateur doit valider